

UNIVERSITI TUNKU ABDUL RAHMAN
Faculty of Information and Communication Technology



UCCD3223 Mobile Applications Development
(Jun 2026 Trimester)

Individual Practical Assignment

Name	CHAI BOON HONG
Student ID	22ACB06806
Course	Communications and Networking (CN)
Practical Group	P1
Lecturer	Mr. Tan Chiang Kang

Marking scheme	Marks	Remarks
Correctness	× 2.5	
Design	× 3.5	
User Friendliness	× 2	
Neat Program Documentation		
Report Format		
TOTAL		

Contents

1.0 Brief Account of the Mobile Application	1
2.0 Application Flow and Play Modes	1
3.0 User Guide and Screen Explanation	2
4.0 Coverage of the Four Required Topics	12
5.0 Additional Features	13
6.0 Techniques and Components Used	14
7.0 Program Design, Correctness and Testing	14
Appendix A - Java Source Code	16

Reading guide: Section 3 explains every screenshot and user action. Sections 4 to 7 connect the finished application to the assignment requirements, implementation techniques and verification evidence.

1.0 Brief Account of the Mobile Application

Number Quest is an offline Android learning game for young children. It provides random exercises for number-to-object association, place value, number recognition and number sequencing. The child can open any topic directly from the home screen, practise without a time limit in Fun mode, or choose a challenge mode with score and timer feedback. The interface uses large controls, plain instructions, encouraging feedback and a consistent blue Material 3 design.

Item	Current configuration
Application	Number Quest - com.uccd3223.p1_chai_boon_hong_2206806
Platform	Android; minimum API 28, compile/target API 36
Implementation	Java 17 activities, Android XML views and Material 3 components
Learning range	Whole numbers from 0 to 99; single-digit object quantities
Operation	Fully local and offline; no account, advertising, analytics or child data collection
Feedback	Visual feedback, optional vibration, background music and short sound effects

Design objective: Each screen keeps the child focused on one clear action while preserving direct access to all four required learning topics.

2.0 Application Flow and Play Modes

The home screen is the navigation centre. The child first chooses Fun or Challenge, then selects one of the four topic cards. Challenge opens a second choice for Time Attack or Round Rush. During play, the game presents a random exercise, gives immediate feedback and updates the current score. Pause, Settings, results and the Trophy Shelf use the same rounded visual language as the main screens.

Mode	Flow and scoring
Fun	Open any topic and practise freely. Correct answers add stars. A session with at least one correct answer can be saved; a zero-score session is not stored.
Time Attack	Choose 60, 90 or 120 seconds. Score as many correct answers as possible before the exact remaining time reaches zero. Each duration has a separate best score.
Round Rush	Complete a fixed set of questions per round. Progress is shown as the current round and question count, and the highest completed round is stored separately.

2.1 Main user flow

- Open Number Quest and choose a mode.
- Select Count & Match, Tens & Ones, Word Detective or Number Order.
- Read the short instruction, answer once, and receive correct or retry feedback.
- Use Pause when a break, Settings change or return to Home is needed.
- Review best scores and previous results on the Trophy Shelf.

2.2 Safe back and pause behaviour

Every game header contains a labelled 52 dp Pause control. Android's Back action uses the same safe flow when progress exists. If the current score is zero, Back or Home exits immediately because there is nothing to save. When a Fun score is above zero, the app explains that the stars can be saved before leaving. An incomplete challenge is never written as a completed result or high score.

3.0 User Guide and Screen Explanation

3.1 Home screen and direct topic access

Figure 1 shows the current home screen. The mode selector appears above the four topic cards, so every required topic is directly reachable. The 52 dp gear button opens Settings, while the Trophy button opens saved results. The large cards combine text, colour and illustrations so meaning does not depend on colour alone.

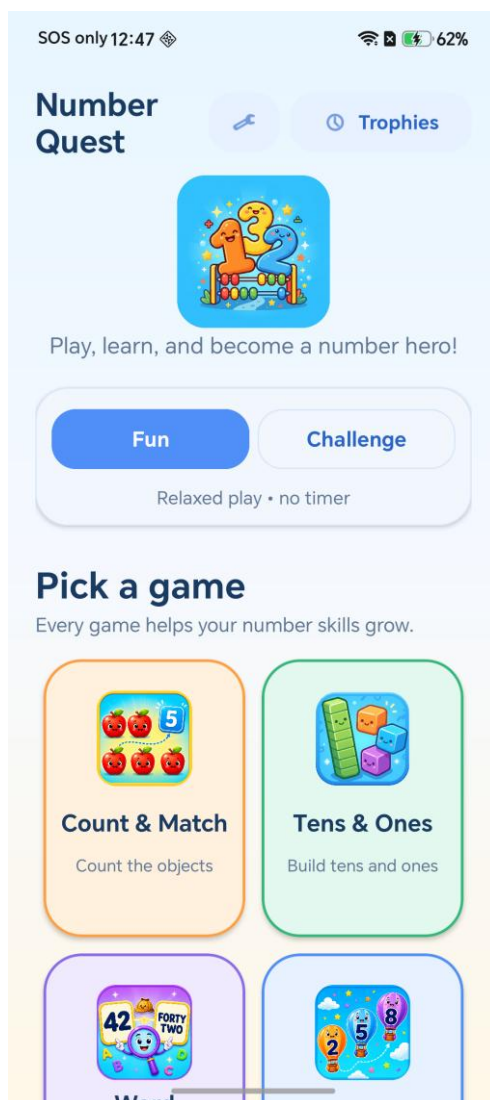


Figure 1: Current home screen with Settings, Trophy Shelf and four topic cards

3.2 Choosing Challenge mode

Figures 2 to 4 show the Challenge setup. Selecting Challenge reveals Time Attack and Round Rush without hiding the topic cards. Time Attack then offers 60, 90 or 120 seconds. These durations are separate score categories, which prevents a longer session from being compared unfairly with a shorter one.

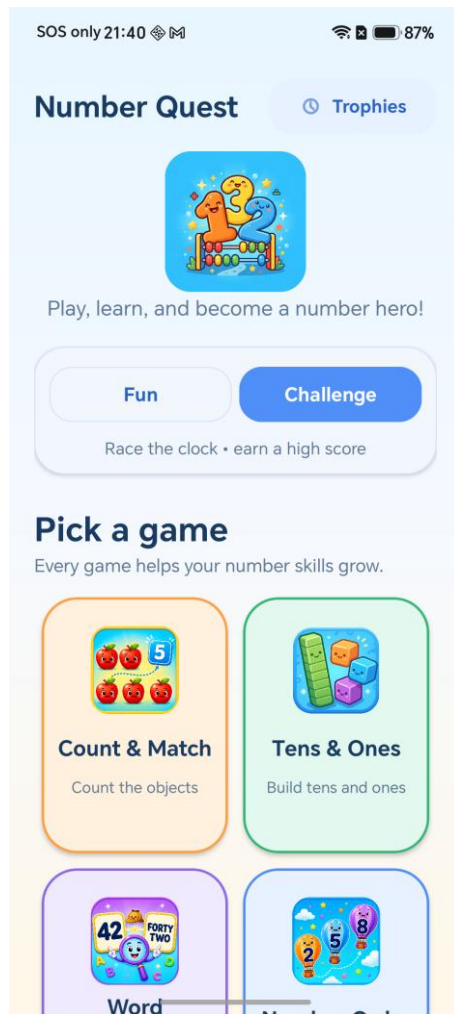


Figure 2: Challenge selected

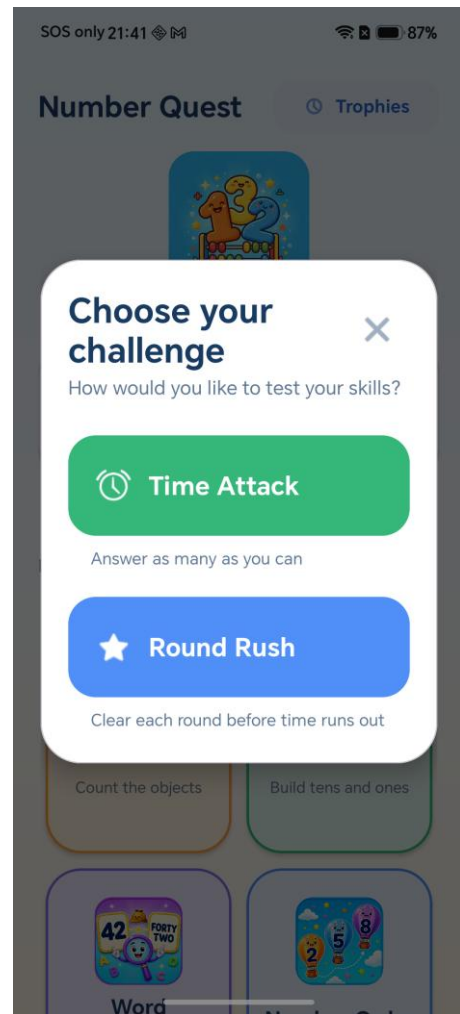


Figure 3: Time Attack and Round Rush choices

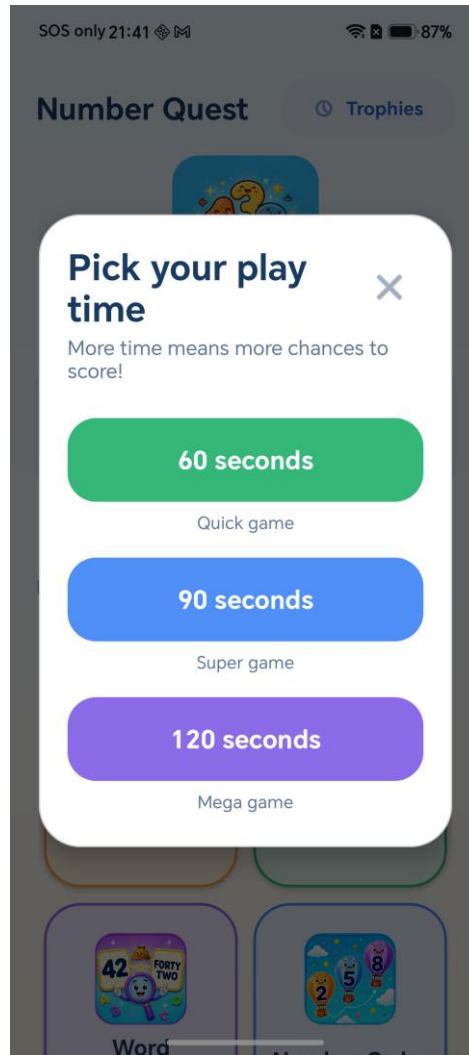


Figure 4: Time Attack duration selection

3.3 Count & Match - number-to-object association

Figure 5 presents a random single-digit quantity of familiar objects. The child counts the objects and taps the matching numeral. The header shows the current stars and the best score for the selected mode category. Figure 6 demonstrates the positive response: the selected choice is confirmed visually, one correct sound is played, and the question advances only once.

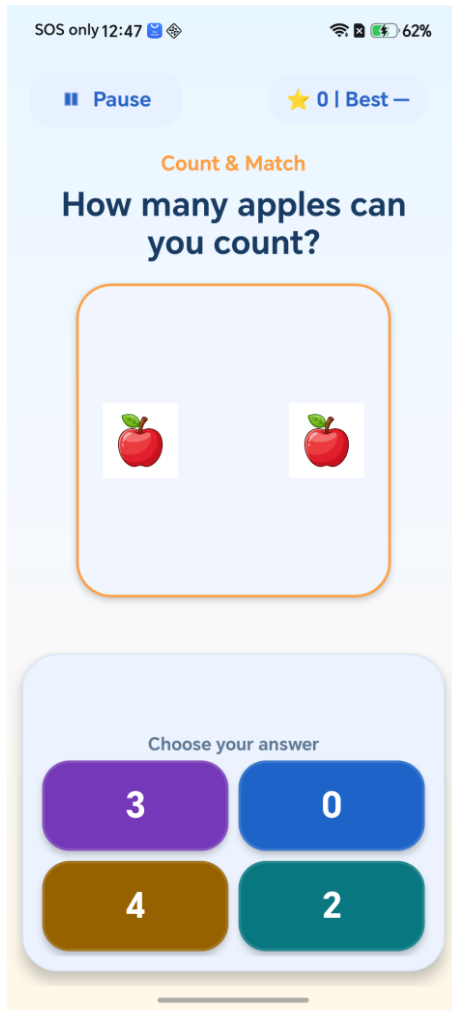


Figure 5: Count & Match with Pause and current/best score

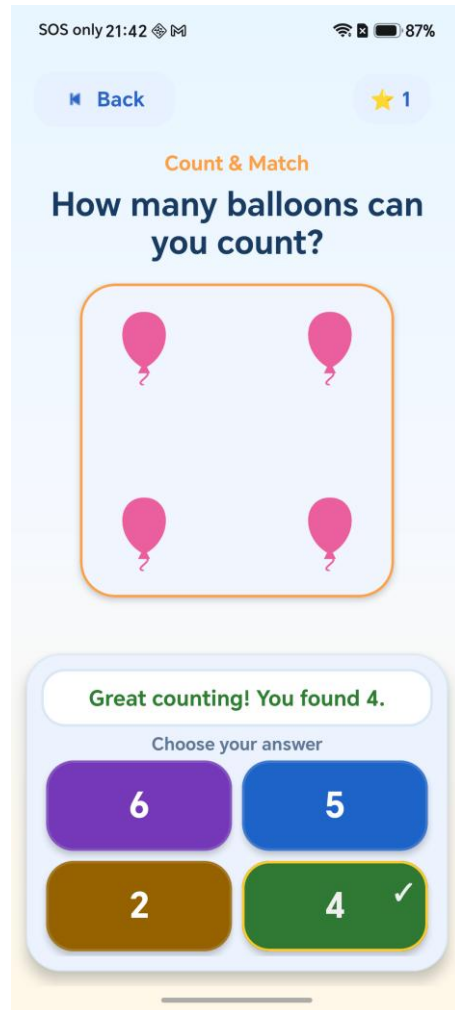


Figure 6: Immediate correct-answer feedback

3.4 Tens & Ones - place value

Figure 7 teaches a two-digit whole number through grouped tens and separate ones. The instruction uses child-friendly words instead of unexplained addition or equality symbols. A new number and answer positions are generated for each exercise, while validation keeps every value in the required 0 to 99 range.

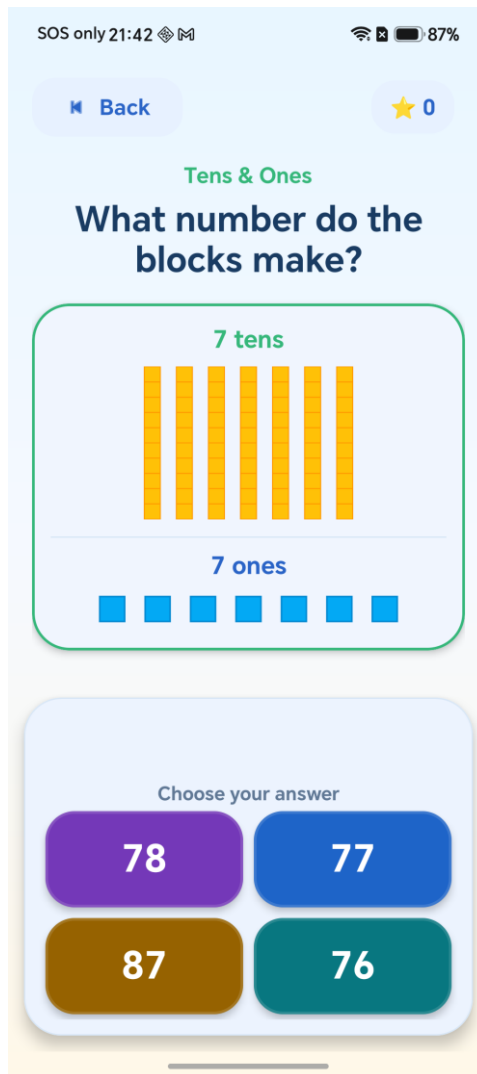


Figure 7: Tens & Ones visual place-value exercise

3.5 Word Detective - number recognition

Word Detective practises both directions of the same association. In Figure 8, the child reads an English number word and chooses its numeral. In Figure 9, the child sees a numeral and chooses the matching English words. Each exercise contains one correct answer and unique alternatives for numbers no greater than two digits.

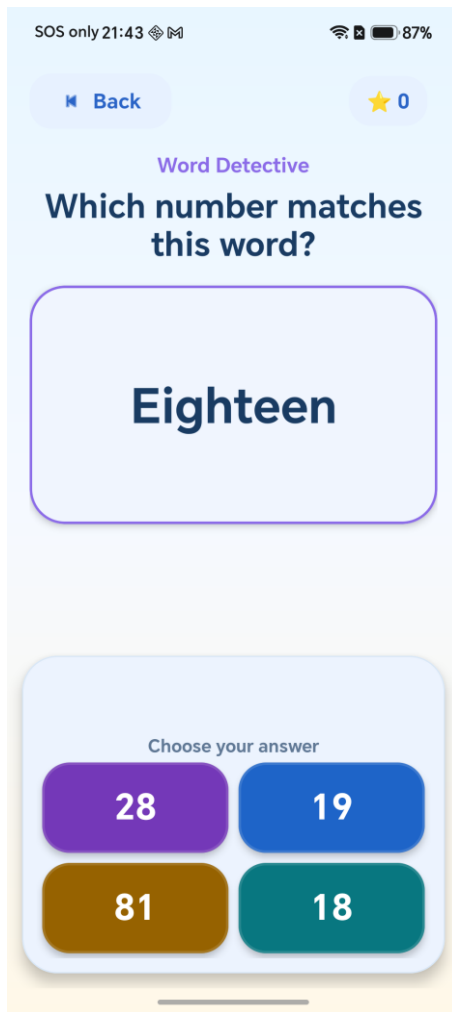


Figure 8: English words to numeral

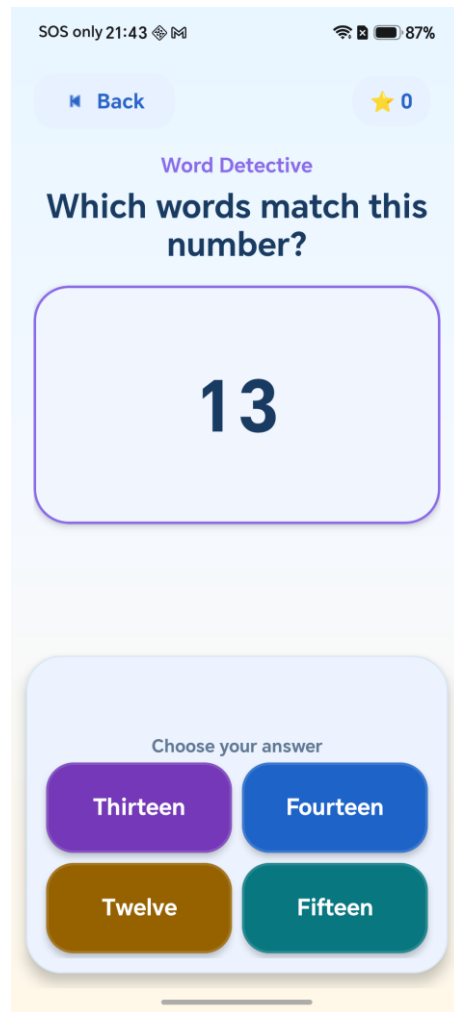


Figure 9: Numeral to English words

3.6 Number Order - numbers in sequence

Figures 10 and 11 show Number Order. The main instruction remains 'Put the numbers in order', while a persistent bold label states either 'Smallest to biggest' or 'Biggest to smallest'. The label stays visible as numbers are placed. Children can drag a number or use the tap alternative, and an incorrect attempt gives a specific hint about the next smallest or biggest value.

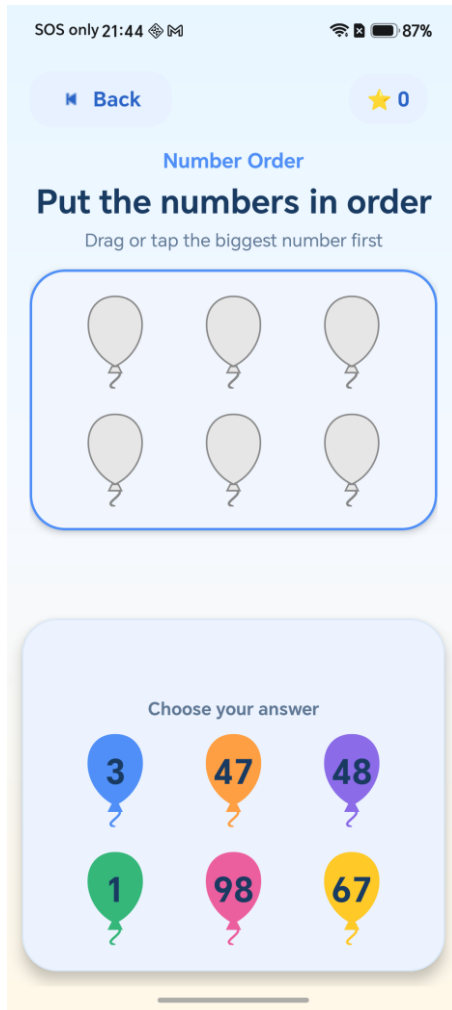


Figure 10: Sequence direction and available numbers

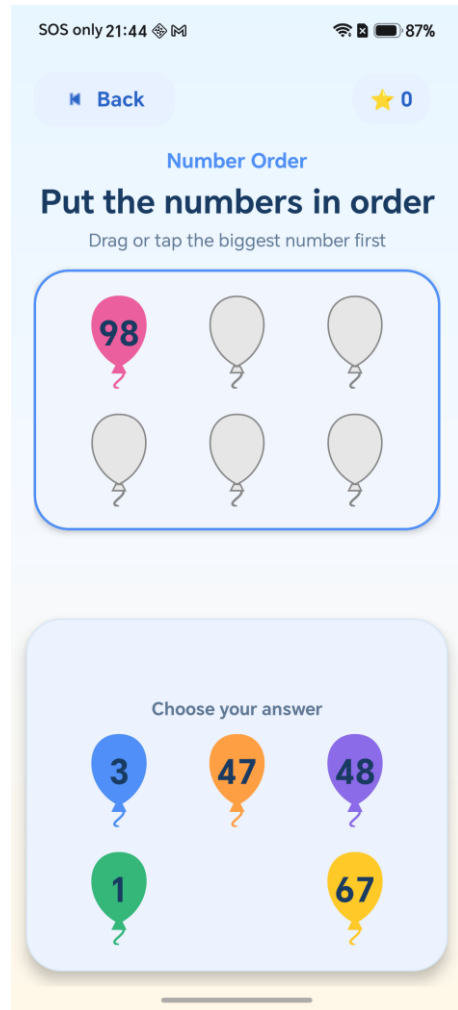


Figure 11: Numbers being placed in sequence

3.7 Results

At the end of a completed challenge, the result dialog stops further scoring and clearly reports the outcome. Figure 12 shows the same full-screen rounded dialog treatment used elsewhere. The child can return Home or try again. Rotation cannot create a second history entry because completion and saved-result state are retained.

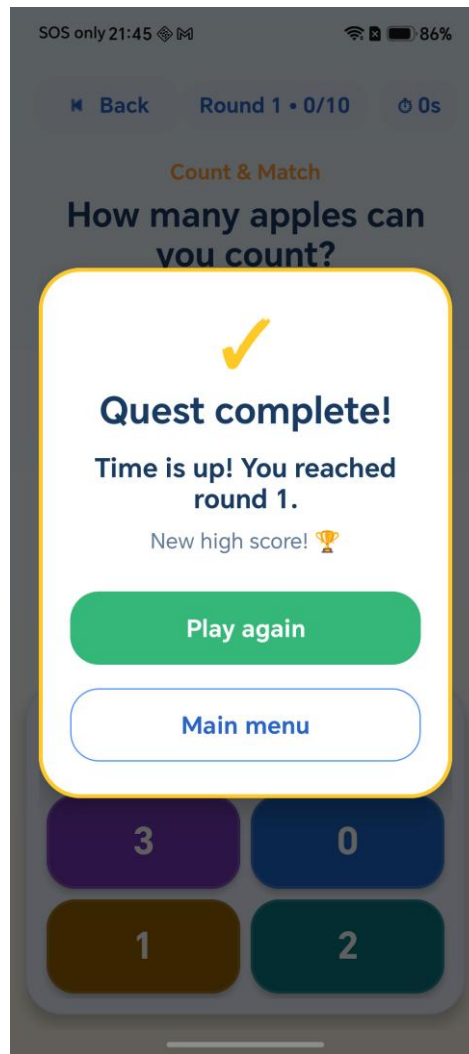


Figure 12: Challenge result dialog

3.8 Settings, music and feedback

Figure 13 shows the scrollable Settings screen. Music and sound effects each have an enable switch and a 0 to 100 volume slider; their defaults are 30% and 80%. Vibration feedback is enabled by default and can be disabled without requesting the VIBRATE permission. Changes take effect immediately and remain local. Reset progress and settings requires confirmation before clearing scores, history and preferences.

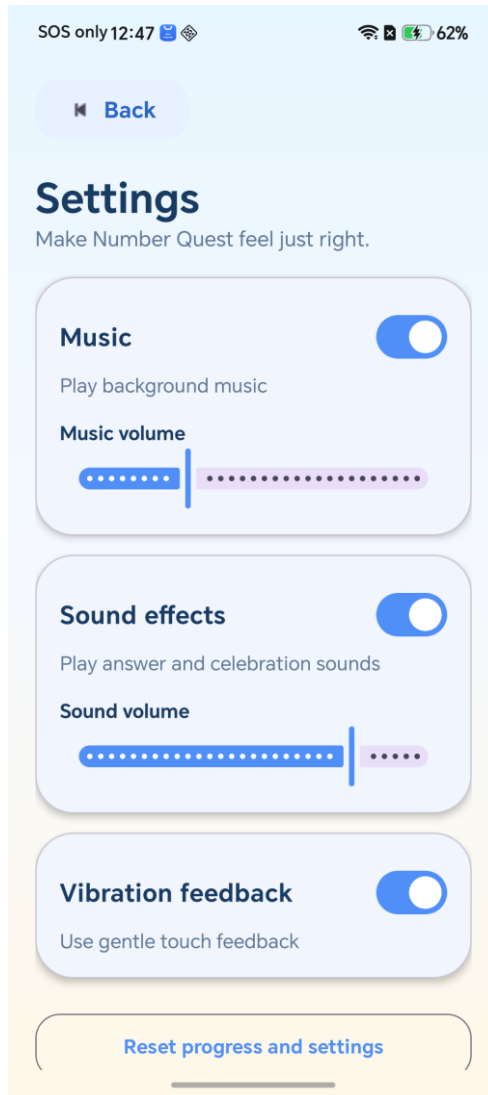


Figure 13: Music, sound effects, vibration and reset controls

3.9 Pause menu and safe navigation

Figure 14 shows the shared pause menu used by all four games. Resume returns to the exact exercise, Settings opens without restarting play, and Home follows the score-sensitive exit rule. Pausing blocks answer taps and Number Order movement, freezes challenge timers and preserves any delayed next-question transition. If the app enters the background during a game, it returns to this menu.

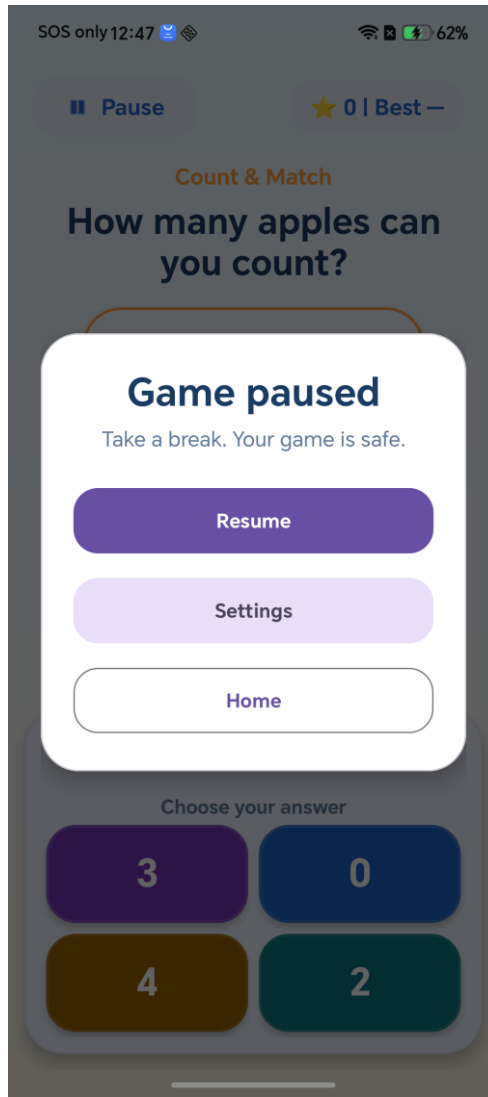


Figure 14: Consistent Resume, Settings and Home pause menu

3.10 Best scores and Trophy Shelf ranking

Figure 15 displays saved history newest first. A gold vector trophy marks only the highest result for each game and fair category; earlier results use a clock icon. If scores tie, the newest tied result receives gold. Fun, each Time Attack duration and Round Rush are ranked independently. Older Time Attack records without duration remain readable but are treated as previous results rather than category winners.

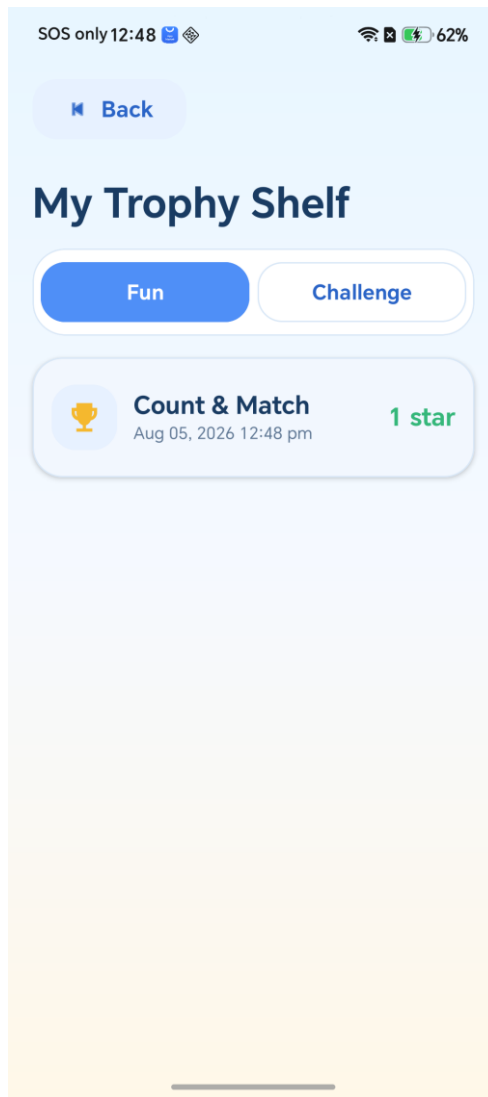


Figure 15: Trophy Shelf with category-aware highest-score icon

4.0 Coverage of the Four Required Topics

Required topic	Implementation in Number Quest	Evidence
Number-to-object association	Count & Match generates a single-digit quantity of objects and unique numeral choices containing exactly one correct answer.	Figures 5-6
Place value	Tens & Ones represents a two-digit whole number with grouped tens and individual ones, supported by plain-language instruction.	Figure 7
Number recognition	Word Detective alternates between words-to-number and number-to-words exercises for values from 0 to 99.	Figures 8-9
Numbers in sequence	Number Order generates unique numbers for ascending or descending arrangement and keeps the chosen direction visible.	Figures 10-11

All four topics can be opened directly from the home screen shown in Figure 1. Their generators randomise exercises and answer positions, validate numeric bounds, avoid duplicate choices and include exactly one correct response. This satisfies the minimum demonstration while the shared modes and feedback improve replay value without replacing the required learning tasks.

5.0 Additional Features

5.1 Fun mode

Fun mode provides open-ended practice without timer pressure. Each correct answer adds one star, and the child may stop at any time. A score of zero is not stored. If at least one star has been earned, the safe Home flow explains that progress can be saved. Fun scores are ranked only against other Fun sessions for the same topic.

5.2 Time Attack

Time Attack offers 60, 90 and 120-second sessions. The countdown uses elapsed time so pause and lifecycle events preserve the exact remaining duration. Each duration has its own best-score category, giving a fair comparison. When time ends, controls are locked before the result dialog is shown, which prevents late taps from changing the final score.

5.3 Round Rush

Round Rush turns practice into a structured sequence of questions and rounds. The header displays the current round, question progress and best completed round. An unfinished Round Rush session is not saved as a completed result. The shared pause controller freezes progression and restores it once, so leaving for Settings or rotating the device cannot skip or duplicate a question.

5.4 Settings, audio and haptics

The Settings screen provides immediate controls for looping background music, short feedback sounds and vibration. Music defaults to 30% and effects to 80%; disabling a switch retains its previous slider value. MediaPlayer handles the application-scoped loop, while SoundPool plays correct, incorrect and new-record effects. Audio pauses only when the whole app backgrounds or loses focus.

5.5 Pause and lifecycle safety

Every topic uses the same Pause menu with Resume, Settings and Home. While paused, taps, score changes, timers, drag-and-drop and delayed transitions are blocked. Returning from Settings shows the pause menu again rather than silently resuming. Pause state is saved across rotation, and a backgrounded game automatically pauses so the child always decides when to continue.

5.6 Best scores and fair ranking

Current and best values appear together in every game header. Best scores are stored separately for Fun, Time Attack 60, 90 and 120 seconds, and Round Rush. The Trophy Shelf awards one gold trophy to the newest highest result in each game/category combination. Previous or legacy results use a clock, making the ranking understandable without relying on colour alone.

5.7 Reset and local persistence

Reset progress and settings is protected by a confirmation dialog. Confirming clears Trophy Shelf history, best scores and preferences, then restores the documented defaults. SharedPreferences and Gson keep data entirely on the device. History is limited to the newest 100 records, and damaged JSON is handled safely so malformed saved data cannot crash the application.

5.8 Accessible, encouraging feedback

Meaningful controls combine icons with text or accessible descriptions, and touch targets are at least 48 dp. Correct and retry states use wording, shapes and sound rather than colour alone. Number Order also supports tapping as an alternative to dragging. Live announcements describe direction and important changes, while optional vibration can be disabled for children who prefer reduced sensory feedback.

6.0 Techniques and Components Used

Technique or component	Purpose in the application
Java 17 activities and XML views	Implements the screens with the assignment's required Android architecture and reusable resource-based layouts.
Material 3 components	Provides consistent rounded buttons, cards, switches, sliders, dialogs and accessible visual hierarchy.
Application.ActivityLifecycleCallbacks	Keeps BGM active across screens but pauses it when the complete application enters the background.
MediaPlayer, SoundPool and audio focus	Loops BGM efficiently and plays low-latency correct, incorrect and new-record effects while respecting system audio focus.
SharedPreferences and Gson	Stores settings, best scores and a bounded local result history without a network service.
CountDownTimer and elapsed-time state	Maintains coherent Time Attack timing through pause, resume and lifecycle recreation.
OnBackPressedDispatcher and shared pause logic	Applies the same score-sensitive navigation and pause behaviour to all topic activities.
RecyclerView and vector drawables	Displays efficient Trophy Shelf history with accessible gold trophy and previous-result clock icons.
Drag-and-drop with tap alternative	Supports playful Number Order interaction while retaining keyboard/tap accessibility.
JUnit unit tests and Android lint	Verify generators, score categories, ranking ties, legacy history and source-quality constraints.

7.0 Program Design, Correctness and Testing

7.1 Reusable architecture

Class/component	Responsibility
BaseGameActivity	Shared mode state, scoring, best-score display, pause flow, timers, delayed transitions, feedback and lifecycle restoration.
ExerciseGeneratorUtil	Focused, unit-testable random exercise generation and number-word conversion.
GameAudioManager	Application-scoped BGM, SoundPool effects, volume updates and audio-focus response.
GameSettingsManager	Persistent music, effects, volume and vibration preferences with documented defaults.
ProgressManager	Central score-category keys, best-score updates and complete reset operation.
HistoryManager / HistoryRankingUtil	Bounded JSON history, legacy-safe parsing and category-aware trophy ranking.
Four topic activities	Topic-specific views and answers built on the same shared game contract.

7.2 Validated random exercise generation

Exercise generation is separated from screen code so its rules can be tested directly. Values remain within 0 to 99, object quantities remain single digit, choices are unique and exactly one choice is correct. Sequence sets contain unique values and a clear direction. Bounded loops and deterministic fallbacks guarantee termination, while remembered previous prompts reduce immediate repetition where the activity promises variety.

7.3 One response, one score change

Each activity locks the current question as soon as an answer is accepted. Rapid repeated taps, drag completion and delayed callbacks therefore cannot count the same correct response twice. Challenge completion disables further input before showing the result. Pending next-question work is represented explicitly and restored once after pause or rotation, preventing duplicated scoring, skipped exercises or multiple history writes.

7.4 Pause, timer and lifecycle testing

The pause state records whether gameplay, a timer or a delayed transition was active and stores the exact remaining duration. While the pause card or Settings is visible, all answer and drag controls remain blocked. Resume reconstructs only the previously active work. Rotation and background/foreground tests verify that the menu remains visible, the timer does not run secretly, and no transition executes twice.

7.5 Persistence, ranking and verification

Unit tests cover score categories, Time Attack durations, best-score replacement, newest-wins ties and legacy records without duration. Reset tests confirm that history, scores and settings return to defaults. Damaged JSON falls back safely and history stays within 100 entries. The final project is verified with `testDebugUnitTest`, `lintDebug` and `assembleDebug`, producing a debug APK with zero test or lint errors.

Appendix A - Java Source Code

This appendix contains the Java source files used by the submitted application. The listings demonstrate the four learning activities, shared game behaviour, audio and settings management, persistence, ranking and validated exercise generation. Layout XML, string resources and bundled audio licences remain in the submitted source tree.

A.1 com/uccd3223/p1_chai_boon_hong_2206806/AnswerChoice.java

```
package com.uccd3223.p1_chai_boon_hong_2206806;

import java.util.Objects;

public final class AnswerChoice {
    private final int value;
    private final String label;
    private final String contentDescription;

    public AnswerChoice(int value, String label, String contentDescription) {
        this.value = value;
        this.label = Objects.requireNonNull(label);
        this.contentDescription = Objects.requireNonNull(contentDescription);
    }

    public int getValue() {
        return value;
    }

    public String getLabel() {
        return label;
    }

    public String getContentDescription() {
        return contentDescription;
    }
}
```

A.2 com/uccd3223/p1_chai_boon_hong_2206806/AssociationActivity.java

```
package com.uccd3223.p1_chai_boon_hong_2206806;

import android.content.res.ColorStateList;
import android.os.Bundle;
import android.view.View;
import android.widget.GridLayout;
import android.widget.ImageView;
import android.widget.TextView;

import com.google.android.material.card.MaterialCardView;
import com.uccd3223.p1_chai_boon_hong_2206806.util.ExerciseGeneratorUtil;

import androidx.core.content.ContextCompat;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.ThreadLocalRandom;

public class AssociationActivity extends BaseGameActivity {
    private enum CountTheme {
        APPLE(R.drawable.game_apple, R.string.count_apples_prompt, R.string.count_object_apple, null),
        STAR(R.drawable.ic_count_star, R.string.count_stars_prompt, R.string.count_object_star, null),
        BALLOON(
            R.drawable.ic_balloon,
            R.string.count_balloons_prompt,
            R.string.count_object_balloon,
            R.color.game_balloon_pink
        );

        final int drawableRes;
        final int promptRes;
        final int objectNameRes;
        final Integer tintRes;

        CountTheme(int drawableRes, int promptRes, int objectNameRes, Integer tintRes) {
            this.drawableRes = drawableRes;
            this.promptRes = promptRes;
            this.objectNameRes = objectNameRes;
            this.tintRes = tintRes;
        }
    }
}
```

```

private static final String STATE_TARGET_COUNT = "association_target_count";
private static final String STATE_LAST_TARGET_COUNT = "association_last_target_count";
private static final String STATE_THEME = "association_theme";
private static final String STATE_LAST_THEME = "association_last_theme";
private static final String STATE_OPTIONS = "association_options";
private static final String STATE_QUESTION_LOCKED = "association_question_locked";

private GridLayout objectsGridLayout;
private GridLayout optionsContainerLayout;
private TextView tvInstruction;
private TextView tvFeedback;
private int targetCount;
private int lastTargetCount = -1;
private CountTheme currentTheme = CountTheme.APPLE;
private int lastThemeOrdinal = -1;
private List<Integer> currentOptions = Collections.emptyList();
private boolean questionLocked;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_association);
    setupGameModeUI();
    initializeUI();
    if (savedInstanceState != null && savedInstanceState.containsKey(STATE_TARGET_COUNT)) {
        restoreQuestion(savedInstanceState);
    } else {
        loadNextQuestion();
    }
}

private void initializeUI() {
    objectsGridLayout = findViewById(R.id.objectsGridLayout);
    optionsContainerLayout = findViewById(R.id.optionsContainerLayout);
    tvInstruction = findViewById(R.id.tvInstruction);
    tvFeedback = findViewById(R.id.tvFeedback);
}

private void loadNextQuestion() {
    questionLocked = false;
    resetFeedback(tvFeedback);
    objectsGridLayout.removeAllViews();
    do {
        targetCount = ExerciseGeneratorUtil.generateObjectCount(9);
        CountTheme[] themes = CountTheme.values();
        currentTheme = themes[ThreadLocalRandom.current().nextInt(themes.length)];
    } while (targetCount == lastTargetCount && currentTheme.ordinal() == lastThemeOrdinal);

    lastTargetCount = targetCount;
    lastThemeOrdinal = currentTheme.ordinal();
    currentOptions = ExerciseGeneratorUtil
        .generateRecognitionOptions(targetCount, 9, 4)
        .getOptions();
    renderQuestion();
}

private void restoreQuestion(Bundle state) {
    targetCount = state.getInt(STATE_TARGET_COUNT);
    lastTargetCount = state.getInt(STATE_LAST_TARGET_COUNT, targetCount);
    int themeIndex = Math.max(0, Math.min(CountTheme.values().length - 1, state.getInt(STATE_THEME)));
    currentTheme = CountTheme.values()[themeIndex];
    lastThemeOrdinal = state.getInt(STATE_LAST_THEME, currentTheme.ordinal());
    questionLocked = state.getBoolean(STATE_QUESTION_LOCKED);
    ArrayList<Integer> restoredOptions = state.getIntegerArrayList(STATE_OPTIONS);
    currentOptions = restoredOptions == null ? Collections.emptyList() : restoredOptions;
    if (currentOptions.size() != 4) {
        currentOptions = ExerciseGeneratorUtil
            .generateRecognitionOptions(targetCount, 9, 4)
            .getOptions();
    }
    renderQuestion();
    if (questionLocked) {
        restoreCompletedAnswerChoice(
            optionsContainerLayout,
            targetCount,
            tvFeedback,
            getString(R.string.count_success, targetCount),
            this::loadNextQuestion
        );
    }
}

private void renderQuestion() {
    tvInstruction.setText(currentTheme.promptRes);
    renderObjectsUI();
    renderOptionsUI();
}

private void renderObjectsUI() {
    objectsGridLayout.removeAllViews();
}

```

```

float density = getResources().getDisplayMetrics().density;
int size = (int) (60 * density);
int margin = (int) (7 * density);
List<Integer> activeIndices = getActiveIndices(targetCount);

for (int index = 0; index < 9; index++) {
    boolean active = activeIndices.contains(index);
    ImageView itemView = new ImageView(this);
    GridLayout.LayoutParams params = new GridLayout.LayoutParams();
    params.width = size;
    params.height = size;
    params.setMargins(margin, margin, margin, margin);
    itemView.setLayoutParams(params);
    itemView.setScaleType(ImageView.ScaleType.FIT_CENTER);
    if (active) {
        itemView.setImageResource(currentTheme.drawableRes);
        if (currentTheme.tintRes != null) {
            itemView.setImageTintList(ColorStateList.valueOf(
                ContextCompat.getColor(this, currentTheme.tintRes)
            ));
        }
        itemView.setContentDescription(getString(currentTheme.objectNameRes));
        itemView.setImportantForAccessibility(View.IMPORTANT_FOR_ACCESSIBILITY_YES);
    } else {
        itemView.setVisibility(View.INVISIBLE);
        itemView.setImportantForAccessibility(View.IMPORTANT_FOR_ACCESSIBILITY_NO);
    }
    objectsGridLayout.addView(itemView);
}

private List<Integer> getActiveIndices(int count) {
    switch (count) {
        case 1: return Collections.singletonList(4);
        case 2: return Arrays.asList(3, 5);
        case 3: return Arrays.asList(3, 4, 5);
        case 4: return Arrays.asList(0, 2, 6, 8);
        case 5: return Arrays.asList(0, 2, 4, 6, 8);
        case 6: return Arrays.asList(0, 2, 3, 5, 6, 8);
        case 7: return Arrays.asList(0, 2, 3, 4, 5, 6, 8);
        case 8: return Arrays.asList(0, 1, 2, 3, 5, 6, 7, 8);
        case 9: return Arrays.asList(0, 1, 2, 3, 4, 5, 6, 7, 8);
        default: return Collections.emptyList();
    }
}

private void renderOptionsUI() {
    List<AnswerChoice> choices = new ArrayList<>();
    for (int option : currentOptions) {
        choices.add(new AnswerChoice(
            option,
            getString(R.string.number_value, option),
            getString(R.string.answer_number, option)
        ));
    }
    renderAnswerChoices(optionsContainerLayout, choices, (choice, card) ->
        checkAnswer(choice.getValue(), card));
}

private void checkAnswer(int selectedOption, MaterialCardView card) {
    if (questionLocked || !canAcceptInput()) return;
    if (selectedOption == targetCount) {
        questionLocked = true;
        completeAnswerChoice(
            optionsContainerLayout,
            card,
            tvFeedback,
            getString(R.string.count_success, targetCount),
            this::loadNextQuestion
        );
    } else {
        retryAnswerChoice(card, tvFeedback, getString(R.string.count_retry));
    }
}

@Override
protected void onSaveInstanceState(Bundle outState) {
    outState.putInt(STATE_TARGET_COUNT, targetCount);
    outState.putInt(STATE_LAST_TARGET_COUNT, lastTargetCount);
    outState.putInt(STATE_THEME, currentTheme.ordinal());
    outState.putInt(STATE_LAST_THEME, lastThemeOrdinal);
    outState.putIntegerArrayList(STATE_OPTIONS, new ArrayList<>(currentOptions));
    outState.putBoolean(STATE_QUESTION_LOCKED, questionLocked);
    super.onSaveInstanceState(outState);
}
}

```

A.3 com/uccd3223/p1_chai_boon_hong_2206806/BalloonFrameLayout.java

```
package com.uccd3223.p1_chai_boon_hong_2206806;
```

```
import android.content.Context;
```

```

import android.util.AttributeSet;
import android.widget.FrameLayout;

public class BalloonFrameLayout extends FrameLayout {
    public BalloonFrameLayout(Context context) {
        super(context);
    }

    public BalloonFrameLayout(Context context, AttributeSet attrs) {
        super(context, attrs);
    }

    public BalloonFrameLayout(Context context, AttributeSet attrs, int defStyleAttr) {
        super(context, attrs, defStyleAttr);
    }

    @Override
    public boolean performClick() {
        super.performClick();
        return true;
    }
}

```

A.4 com/uccd3223/p1_chai_boon_hong_2206806/BaseGameActivity.java

```

package com.uccd3223.p1_chai_boon_hong_2206806;

import android.app.Dialog;
import android.content.Intent;
import android.content.res.Configuration;
import android.os.Bundle;
import android.os.CountDownTimer;
import android.os.SystemClock;
import android.util.TypedValue;
import android.view.HapticFeedbackConstants;
import android.view.View;
import android.view.ViewGroup;
import android.view.Window;
import android.view.animation.AnimationUtils;
import android.widget.Button;
import android.widget.GridLayout;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.activity.OnBackPressedCallback;
import androidx.core.content.ContextCompat;

import com.google.android.material.card.MaterialCardView;

import java.util.Arrays;
import java.util.List;

public class BaseGameActivity extends AppCompatActivity {
    protected interface AnswerSelectionListener { void onSelected(AnswerChoice choice, MaterialCardView card); }

    private static final String STATE_SCORE = "state_score";
    private static final String STATE_ROUND = "state_round";
    private static final String STATE_ROUND_SCORE = "state_round_score";
    private static final String STATE_TIME_REMAINING = "state_time_remaining";
    private static final String STATE_HISTORY_SAVED = "state_history_saved";
    private static final String STATE_GAME_OVER = "state_game_over";
    private static final String STATE_RESULT_MESSAGE = "state_result_message";
    private static final String STATE_RESULT_RECORD = "state_result_record";
    private static final String STATE_PAUSED = "state_paused";
    private static final String STATE_PENDING_DELAY = "state_pending_delay";
    private static final String STATE_RECORD_CELEBRATED = "state_record_celebrated";

    protected String gameMode = "FUN";
    protected int score;
    protected boolean isGameOver;

    private TextView tvScore;
    private TextView tvTimer;
    private CountDownTimer countDownTimer;
    private long timeLimitMs = 60_000L;
    private long remainingTimeMs;
    private long timerDeadline;
    private int currentRound = 1;
    private int targetQuestionsForRound = 10;
    private int timeLimitForRoundSeconds = 30;
    private int roundScore;
    private boolean historySaved;
    private boolean isGamePaused;
    private boolean recordCelebrated;
    private String pendingResultMessage;
    private String pendingRecordText;
    private Dialog resultDialog;

```

```

private Dialog pauseDialog;
private Runnable pendingTransition;
private View pendingHost;
private long pendingTransitionDue;
private long restoredPendingDelay;

@Override protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    String requestedMode = getIntent().getStringExtra("GAME_MODE");
    gameMode = requestedMode == null ? "FUN" : requestedMode;
    timeLimitMs = getIntent().getLongExtra("TIME_LIMIT", 60_000L);
    if (savedInstanceState != null) {
        score = savedInstanceState.getInt(STATE_SCORE);
        currentRound = savedInstanceState.getInt(STATE_ROUND, 1);
        roundScore = savedInstanceState.getInt(STATE_ROUND_SCORE);
        remainingTimeMs = savedInstanceState.getLong(STATE_TIME_REMAINING);
        historySaved = savedInstanceState.getBoolean(STATE_HISTORY_SAVED);
        isGameOver = savedInstanceState.getBoolean(STATE_GAME_OVER);
        pendingResultMessage = savedInstanceState.getString(STATE_RESULT_MESSAGE);
        pendingRecordText = savedInstanceState.getString(STATE_RESULT_RECORD);
        isGamePaused = savedInstanceState.getBoolean(STATE_PAUSED);
        restoredPendingDelay = savedInstanceState.getLong(STATE_PENDING_DELAY);
        recordCelebrated = savedInstanceState.getBoolean(STATE_RECORD_CELEBRATED);
    }
    getOnBackPressedDispatcher().addCallback(this, new OnBackPressedCallback(true) {
        @Override public void handleOnBackPressed() {
            if (shouldConfirmExit(score)) pauseGame(); else finish();
        }
    });
}

protected void setupGameModeUI() {
    tvScore = findViewById(R.id.tvScore);
    tvTimer = findViewById(R.id.tvTimer);
    findViewById(R.id.btnPause).setOnClickListener(view -> pauseGame());
    targetQuestionsForRound = 10 + (currentRound - 1) * 5;
    timeLimitForRoundSeconds = 30 + (currentRound - 1) * 10;
    tvScore.setVisibility(View.VISIBLE);
    tvTimer.setVisibility("FUN".equals(gameMode) ? View.GONE : View.VISIBLE);
    updateScoreDisplay();

    if (isGameOver) {
        updateTimerText(0L);
        tvScore.post(() -> showResultDialog(pendingResultMessage, pendingRecordText));
        return;
    }
    if ("TIME_ATTACK".equals(gameMode)) {
        long duration = remainingTimeMs > 0 ? remainingTimeMs : timeLimitMs;
        remainingTimeMs = duration;
        updateTimerText((duration + 999L) / 1_000L);
        if (!isGamePaused) startGameTimer(duration);
    } else if ("SCORE_ATTACK".equals(gameMode)) {
        long duration = remainingTimeMs > 0 ? remainingTimeMs : timeLimitForRoundSeconds * 1_000L;
        remainingTimeMs = duration;
        updateTimerText((duration + 999L) / 1_000L);
        if (!isGamePaused) startGameTimer(duration);
    }
}

protected boolean canAcceptInput() { return !isGamePaused && !isGameOver; }

protected boolean onQuestionCompleted() {
    if (!canAcceptInput()) return false;
    boolean playedRecord = false;
    score++;
    if ("SCORE_ATTACK".equals(gameMode)) {
        roundScore++;
        if (roundScore >= targetQuestionsForRound) {
            cancelTimerPreservingTime();
            currentRound++;
            roundScore = 0;
            int seconds = 30 + (currentRound - 1) * 10;
            Toast.makeText(this, getResources().getQuantityString(R.plurals.next_round, seconds, currentRound, seconds),
                Toast.LENGTH_SHORT).show();
            targetQuestionsForRound = 10 + (currentRound - 1) * 5;
            timeLimitForRoundSeconds = seconds;
            startGameTimer(seconds * 1_000L);
        }
    } else if ("FUN".equals(gameMode)) {
        boolean newRecord = ProgressManager.updateBest(this, getClass().getSimpleName(), gameMode, timeLimitMs, score);
        if (newRecord && !recordCelebrated) {
            recordCelebrated = true;
            GameAudioManager.get(this).playRecord();
            playedRecord = true;
        }
    }
    updateScoreDisplay();
    return playedRecord;
}

private int currentBestDisplay() {
    int saved = ProgressManager.getBest(this, getClass().getSimpleName(), gameMode, timeLimitMs);
    int active = "SCORE_ATTACK".equals(gameMode) ? currentRound : score;
}

```

```

        return Math.max(saved, active);
    }

    private void updateScoreDisplay() {
        int saved = ProgressManager.getBest(this, getClass().getSimpleName(), gameMode, timeLimitMs);
        int shownBest = currentBestDisplay();
        String best = saved == ProgressManager.NO_SCORE && shownBest == 0
            ? getString(R.string.best_none)
            : ("SCORE_ATTACK".equals(gameMode) ? getString(R.string.best_round, shownBest) : String.valueOf(shownBest));
        if ("SCORE_ATTACK".equals(gameMode)) {
            tvScore.setText(getString(R.string.round_with_best, currentRound, roundScore, targetQuestionsForRound, best));
        } else {
            tvScore.setText(getString(R.string.score_with_best, score, best));
        }
    }

    protected void renderAnswerChoices(GridLayout container, List<AnswerChoice> choices, AnswerSelectionListener listener) {
        container.removeAllViews();
        int columns = getResources().getConfiguration().orientation == Configuration.ORIENTATION_LANDSCAPE ? choices.size() : 2;
        container.setColumnCount(columns);
        container.setRowCount((choices.size() + columns - 1) / columns);
        List<Integer> colors = Arrays.asList(R.color.answer_purple, R.color.answer_blue, R.color.answer_gold, R.color.answer_teal);
        int gap = getResources().getDimensionPixelSize(R.dimen.answer_option_gap);
        for (int index = 0; index < choices.size(); index++) {
            AnswerChoice choice = choices.get(index);
            MaterialCardView card = (MaterialCardView) getLayoutInflater().inflate(R.layout.item_answer_chocier, container, false);
            card.setCardBackgroundColor(ContextCompat.getColor(this, colors.get(index % colors.size())));
            card.setTag(choice.getValue());
            card.setContentDescription(choice.getContentDescription());
            TextView label = card.findViewById(R.id.answerChocierText);
            label.setText(choice.getLabel());
            boolean hasLetter = choice.getLabel().chars().anyMatch(Character::isLetter);
            label.setTextSize(TypedValue.COMPLEX_UNIT_SP, hasLetter && choice.getLabel().length() >= 11 ? 16f : hasLetter ? 18f : 28f);
            GridLayout.LayoutParams params = new GridLayout.LayoutParams(GridLayout.spec(index / columns), GridLayout.spec(index %
columns, 1f));
            params.width = 0;
            params.height = getResources().getDimensionPixelSize(R.dimen.answer_option_min_height);
            params.setMargins(gap / 2, gap / 2, gap / 2, gap / 2);
            card.setLayoutParams(params);
            card.setOnClickListener(view -> { if (canAcceptInput()) listener.onSelected(choice, card); });
            container.addView(card);
        }
    }

    protected void completeAnswerChoice(GridLayout container, MaterialCardView selectedCard, TextView feedback, String message,
Runnable nextQuestion) {
        if (!canAcceptInput()) return;
        for (int i = 0; i < container.getChildCount(); i++) container.getChildAt(i).setEnabled(false);
        selectedCard.setCardBackgroundColor(ContextCompat.getColor(this, R.color.answer_correct));
        selectedCard.setStrokeColor(ContextCompat.getColor(this, R.color.game_star));
        selectedCard.<TextView>findViewById(R.id.answerStateIcon).setVisibility(View.VISIBLE);
        celebrateQuestion(selectedCard, feedback, message, nextQuestion);
    }

    protected void retryAnswerChoice(MaterialCardView card, TextView feedback, String message) {
        if (!canAcceptInput()) return;
        int originalColor = card.getCardBackgroundColor().getDefaultColor();
        card.setCardBackgroundColor(ContextCompat.getColor(this, R.color.answer_retry));
        showRetryFeedback(card, feedback, message);
        card.postDelayed(() -> { if (card.isEnabled() && !isFinishing()) card.setCardBackgroundColor(originalColor); }, 550L);
    }

    protected void restoreCompletedAnswerChoice(GridLayout container, int correctValue, TextView feedback, String message, Runnable
nextQuestion) {
        for (int i = 0; i < container.getChildCount(); i++) {
            View child = container.getChildAt(i);
            if (child instanceof MaterialCardView) {
                MaterialCardView card = (MaterialCardView) child;
                card.setEnabled(false);
                if (Integer.valueOf(correctValue).equals(card.getTag())) {
                    card.setCardBackgroundColor(ContextCompat.getColor(this, R.color.answer_correct));
                    card.setStrokeColor(ContextCompat.getColor(this, R.color.game_star));
                    card.<TextView>findViewById(R.id.answerStateIcon).setVisibility(View.VISIBLE);
                }
            }
        }
        restoreCompletedQuestion(feedback, message, nextQuestion);
    }

    protected void restoreCompletedQuestion(TextView feedback, String message, Runnable nextQuestion) {
        showPositiveFeedback(feedback, message);
        if (!isGameOver) scheduleTransition(feedback, nextQuestion, restoredPendingDelay > 0 ? restoredPendingDelay : 500L);
        restoredPendingDelay = 0L;
    }

    protected void celebrateQuestion(View anchor, TextView feedback, String message, Runnable nextQuestion) {
        if (!canAcceptInput()) return;
        showPositiveFeedback(feedback, message);
        feedback.setAlpha(0f); feedback.setScaleX(.82f); feedback.setScaleY(.82f);
        feedback.animate().alpha(1f).scaleX(1f).scaleY(1f).setDuration(220L).start();
        if (GameSettingsManager.isVibrationEnabled(this)) anchor.performHapticFeedback(HapticFeedbackConstants.VIRTUAL_KEY);
    }

```

```

        anchor.animate().scaleX(1.06f).scaleY(1.06f).setDuration(140L).withEndAction(() ->
anchor.animate().scaleX(1f).scaleY(1f).setDuration(140L).start()).start();
        if (!onQuestionCompleted()) GameAudioManager.get(this).playCorrect();
        scheduleTransition(feedback, nextQuestion, "FUN".equals(gameMode) ? 900L : 650L);
    }

    protected void showRetryFeedback(View anchor, TextView feedback, String message) {
        feedback.setText(message);
        feedback.setAccessibilityLiveRegion(View.ACCESSIBILITY_LIVE_REGION_POLITE);
        feedback.setTextColor(ContextCompat.getColor(this, R.color.answer_retry));
        feedback.setVisibility(View.VISIBLE);
        if (GameSettingsManager.isVibrationEnabled(this)) anchor.performHapticFeedback(HapticFeedbackConstants.CLOCK_TICK);
        GameAudioManager.get(this).playIncorrect();
        anchor.startAnimation(AnimationUtils.loadAnimation(this, R.anim.shake));
    }

    private void showPositiveFeedback(TextView feedback, String message) {
        feedback.setText(message);
        feedback.setAccessibilityLiveRegion(View.ACCESSIBILITY_LIVE_REGION_POLITE);
        feedback.setTextColor(ContextCompat.getColor(this, R.color.answer_correct));
        feedback.setVisibility(View.VISIBLE);
    }

    protected void resetFeedback(TextView feedback) {
        feedback.clearAnimation(); feedback.setVisibility(View.INVISIBLE); feedback.setAlpha(1f); feedback.setScaleX(1f);
        feedback.setScaleY(1f);
    }

    private void scheduleTransition(View host, Runnable runnable, long delay) {
        clearPendingTransition();
        pendingHost = host;
        pendingTransition = () -> {
            Runnable action = pendingTransition;
            clearPendingTransition();
            if (!isGameOver && !isFinishing() && !isGamePaused && action != null) runnable.run();
        };
        pendingTransitionDue = isGamePaused ? delay : SystemClock.elapsedRealtime() + delay;
        if (!isGamePaused) host.postDelayed(pendingTransition, delay);
    }

    private void clearPendingTransition() {
        if (pendingHost != null && pendingTransition != null) pendingHost.removeCallbacks(pendingTransition);
        pendingHost = null; pendingTransition = null; pendingTransitionDue = 0L;
    }

    private long pendingDelay() {
        if (pendingTransition == null) return 0L;
        return isGamePaused ? Math.max(1L, pendingTransitionDue)
            : Math.max(1L, pendingTransitionDue - SystemClock.elapsedRealtime());
    }

    private void updateTimerText(long seconds) { if (tvTimer != null) tvTimer.setText(getString(R.string.time_format, seconds)); }

    private void startGameTimer(long durationMs) {
        if (isGamePaused || isGameOver) return;
        if (countDownTimer != null) countDownTimer.cancel();
        remainingTimeMs = durationMs;
        timerDeadline = SystemClock.elapsedRealtime() + durationMs;
        countDownTimer = new CountDownTimer(durationMs, 250L) {
            @Override public void onTick(long millis) {
                remainingTimeMs = Math.max(0L, timerDeadline - SystemClock.elapsedRealtime());
                updateTimerText((remainingTimeMs + 999L) / 1_000L);
            }
            @Override public void onFinish() { remainingTimeMs = 0L; updateTimerText(0L); endGame(); }
        }.start();
    }

    private void cancelTimerPreservingTime() {
        if (countDownTimer != null) {
            remainingTimeMs = Math.max(0L, timerDeadline - SystemClock.elapsedRealtime());
            countDownTimer.cancel(); countDownTimer = null;
        }
    }

    protected void pauseGame() {
        if (isGameOver || isFinishing()) return;
        if (!isGamePaused) {
            long transitionDelay = pendingTransition == null ? 0L
                : Math.max(1L, pendingTransitionDue - SystemClock.elapsedRealtime());
            isGamePaused = true;
            cancelTimerPreservingTime();
            if (pendingHost != null && pendingTransition != null) {
                pendingHost.removeCallbacks(pendingTransition);
                pendingTransitionDue = transitionDelay;
            }
        }
        if (hasWindowFocus()) showPauseDialog();
    }

```

```

private void showPauseDialog() {
    if (pauseDialog != null && pauseDialog.isShowing() || isGameOver || isFinishing()) return;
    Dialog dialog = new Dialog(this, R.style.FullScreenDialogTheme);
    pauseDialog = dialog;
    dialog.setContentView(R.layout.dialog_game_pause);
    dialog.setCancelable(false);
    dialog.findViewById(R.id.btnResume).setOnClickListener(view -> resumeGame());
    dialog.findViewById(R.id.btnPauseSettings).setOnClickListener(view -> startActivity(new Intent(this, SettingsActivity.class)));
    dialog.findViewById(R.id.btnPauseHome).setOnClickListener(view -> showHomeConfirmation());
    showFullScreen(dialog);
    dialog.findViewById(R.id.btnResume).sendAccessibilityEvent(android.view.accessibility.AccessibilityEvent.TYPE_VIEW_FOCUSED);
}

private void resumeGame() {
    if (pauseDialog != null) pauseDialog.dismiss();
    pauseDialog = null;
    isGamePaused = false;
    if (!"FUN".equals(gameMode) && remainingTimeMs > 0) startGameTimer(remainingTimeMs);
    if (pendingHost != null && pendingTransition != null) {
        long delay = pendingTransitionDue;
        pendingTransitionDue = SystemClock.elapsedRealtime() + delay;
        pendingHost.postDelayed(pendingTransition, delay);
    }
}

private void showHomeConfirmation() {
    if (!shouldConfirmExit(score)) {
        finish();
        return;
    }
    Dialog dialog = new Dialog(this, R.style.FullScreenDialogTheme);
    dialog.setContentView(R.layout.dialog_confirmation);
    dialog.<TextView>findViewById(R.id.tvConfirmTitle).setText(R.string.leave_game_title);
    dialog.<TextView>findViewById(R.id.tvConfirmMessage).setText(
        "FUN".equals(gameMode) ? R.string.leave_fun_message : R.string.leave_game_message);
    dialog.<Button>findViewById(R.id.btnConfirmPositive).setText(R.string.go_home);
    dialog.<Button>findViewById(R.id.btnConfirmNegative).setText(R.string.keep_playing);
    dialog.findViewById(R.id.btnConfirmPositive).setOnClickListener(view -> { dialog.dismiss(); finish(); });
    dialog.findViewById(R.id.btnConfirmNegative).setOnClickListener(view -> dialog.dismiss());
    showFullScreen(dialog);
}

static boolean shouldConfirmExit(int currentScore) {
    return currentScore > 0;
}

private void endGame() {
    if (isGameOver || isGamePaused) return;
    isGameOver = true;
    cancelTimerPreservingTime(); clearPendingTransition();
    int finalScore = "SCORE_ATTACK".equals(gameMode) ? currentRound : score;
    int previous = ProgressManager.getBest(this, getClass().getSimpleName(), gameMode, timeLimitMs);
    boolean newHigh = ProgressManager.updateBest(this, getClass().getSimpleName(), gameMode, timeLimitMs, finalScore);
    saveHistoryRecord(finalScore);
    if (newHigh && !recordCelebrated) { recordCelebrated = true; GameAudioManager.get(this).playRecord(); }
    pendingRecordText = newHigh ? getString(R.string.new_high_score)
        : getString(R.string.best_score, previous == ProgressManager.NO_SCORE ? finalScore : previous);
    if ("TIME_ATTACK".equals(gameMode)) pendingResultMessage = getResources().getQuantityString(R.plurals.time_attack_result,
score, score);
    else pendingResultMessage = getString(R.string.round_rush_result, currentRound);
    showResultDialog(pendingResultMessage, pendingRecordText);
}

private void showResultDialog(String message, String recordText) {
    if (message == null || recordText == null || isFinishing() || resultDialog != null && resultDialog.isShowing()) return;
    Dialog dialog = new Dialog(this, R.style.FullScreenDialogTheme);
    resultDialog = dialog; dialog.setContentView(R.layout.dialog_game_result); dialog.setCancelable(false);
    dialog.<TextView>findViewById(R.id.tvResultMessage).setText(message);
    dialog.<TextView>findViewById(R.id.tvResultRecord).setText(recordText);
    dialog.<Button>findViewById(R.id.btnMainMenu).setOnClickListener(view -> { dialog.dismiss(); finish(); });
    dialog.<Button>findViewById(R.id.btnPlayAgain).setOnClickListener(view -> { dialog.dismiss(); Intent replay = new
Intent(getIntent()); finish(); startActivity(replay); });
    showFullScreen(dialog);
}

private void showFullScreen(Dialog dialog) {
    dialog.show(); Window window = dialog.getWindow();
    if (window != null) { window.setBackgroundDrawableResource(android.R.color.transparent);
window.setLayout(ViewGroup.LayoutParams.MATCH_PARENT, ViewGroup.LayoutParams.MATCH_PARENT); }
}

private void saveHistoryRecord(int finalScore) {
    if (historySaved) return;
    HistoryManager.saveRecord(this, new HistoryRecord(getClass().getSimpleName().replace("Activity", ""), gameMode, finalScore,
System.currentTimeMillis(), "TIME_ATTACK".equals(gameMode) ? timeLimitMs : 0L));
    historySaved = true;
}

@Override protected void onResume() {
    super.onResume();
    if (isGamePaused && !isGameOver) getWindow().getDecorView().post(this::showPauseDialog);
}

```

```

    }

    @Override protected void onStop() {
        if (!isChangingConfigurations() && !isGameOver) pauseGame();
        super.onStop();
    }

    @Override protected void onDestroy() {
        cancelTimerPreservingTime(); clearPendingTransition();
        if (isFinishing() && "FUN".equals(gameMode) && score > 0) saveHistoryRecord(score);
        super.onDestroy();
    }

    @Override protected void onSaveInstanceState(Bundle out) {
        cancelTimerPreservingTime();
        out.putInt(STATE_SCORE, score); out.putInt(STATE_ROUND, currentRound); out.putInt(STATE_ROUND_SCORE, roundScore);
        out.putLong(STATE_TIME_REMAINING, remainingTimeMs); out.putBoolean(STATE_HISTORY_SAVED, historySaved);
        out.putBoolean(STATE_GAME_OVER, isGameOver); out.putString(STATE_RESULT_MESSAGE, pendingResultMessage);
        out.putString(STATE_RESULT_RECORD, pendingRecordText);
        out.putBoolean(STATE_PAUSED, isGamePaused); out.putLong(STATE_PENDING_DELAY, pendingDelay());
        out.putBoolean(STATE_RECORD_CELEBRATED, recordCelebrated);
        super.onSaveInstanceState(out);
    }
}

```

A.5 com/uccd3223/p1_chai_boon_hong_2206806/GameAudioManager.java

```
package com.uccd3223.p1_chai_boon_hong_2206806;
```

```

import android.content.Context;
import android.media.AudioAttributes;
import android.media.AudioFocusRequest;
import android.media.AudioManager;
import android.media.MediaPlayer;
import android.media.SoundPool;

```

```

public final class GameAudioManager implements AudioManager.OnAudioFocusChangeListener {
    private static GameAudioManager instance;
    private final Context context;
    private final AudioManager audioManager;
    private final SoundPool soundPool;
    private final int correctSound;
    private final int incorrectSound;
    private final int recordSound;
    private MediaPlayer musicPlayer;
    private AudioFocusRequest focusRequest;
    private boolean appForeground;

```

```

    private GameAudioManager(Context context) {
        this.context = context.getApplicationContext();
        audioManager = (AudioManager) context.getSystemService(Context.AUDIO_SERVICE);
        AudioAttributes attributes = new AudioAttributes.Builder()
            .setUsage(AudioAttributes.USAGE_GAME)
            .setContentType(AudioAttributes.CONTENT_TYPE_MUSIC)
            .build();
        soundPool = new SoundPool.Builder().setMaxStreams(2).setAudioAttributes(attributes).build();
        correctSound = soundPool.load(context, R.raw.confirmation_003, 1);
        incorrectSound = soundPool.load(context, R.raw.error_003, 1);
        recordSound = soundPool.load(context, R.raw.power_up_sound_v1, 1);
    }

```

```

    public static synchronized GameAudioManager get(Context context) {
        if (instance == null) instance = new GameAudioManager(context);
        return instance;
    }

```

```

    public void setAppForeground(boolean foreground) {
        appForeground = foreground;
        if (foreground) refreshMusic(); else pauseMusic();
    }

```

```

    public void refreshMusic() {
        if (!appForeground || !GameSettingsManager.isMusicEnabled(context)) {
            pauseMusic();
            return;
        }
        float volume = GameSettingsManager.getMusicVolume(context) / 100f;
        if (musicPlayer == null) {
            musicPlayer = MediaPlayer.create(context, R.raw.title_in_game);
            if (musicPlayer == null) return;
            musicPlayer.setLooping(true);
        }
        musicPlayer.setVolume(volume, volume);
        if (!musicPlayer.isPlaying() && requestAudioFocus()) musicPlayer.start();
    }

```

```

    public void playCorrect() { play(correctSound); }
    public void playIncorrect() { play(incorrectSound); }
    public void playRecord() { play(recordSound); }

```

```

private void play(int soundId) {
    if (!GameSettingsManager.isSoundEnabled(context)) return;
    float volume = GameSettingsManager.getSoundVolume(context) / 100f;
    soundPool.play(soundId, volume, volume, 1, 0, 1f);
}

private boolean requestAudioFocus() {
    AudioAttributes attributes = new AudioAttributes.Builder().setUsage(AudioAttributes.USAGE_GAME).build();
    focusRequest = new AudioFocusRequest.Builder(AudioManager.AUDIOFOCUS_GAIN)
        .setAudioAttributes(attributes).setOnAudioFocusChangeListener(this).build();
    return audioManager.requestAudioFocus(focusRequest) == AudioManager.AUDIOFOCUS_REQUEST_GRANTED;
}

private void pauseMusic() {
    if (musicPlayer != null && musicPlayer.isPlaying()) musicPlayer.pause();
}

@Override public void onAudioFocusChange(int focusChange) {
    if (focusChange == AudioManager.AUDIOFOCUS_GAIN) refreshMusic(); else pauseMusic();
}
}

```

A.6 com/uccd3223/p1_chai_boon_hong_2206806/GameSettingsManager.java

```
package com.uccd3223.p1_chai_boon_hong_2206806;
```

```
import android.content.Context;
import android.content.SharedPreferences;
```

```

public final class GameSettingsManager {
    private static final String PREFS = "NumberQuestSettings";
    private static final String MUSIC_ENABLED = "music_enabled";
    private static final String MUSIC_VOLUME = "music_volume";
    private static final String SOUND_ENABLED = "sound_enabled";
    private static final String SOUND_VOLUME = "sound_volume";
    private static final String VIBRATION_ENABLED = "vibration_enabled";

    public static final int DEFAULT_MUSIC_VOLUME = 30;
    public static final int DEFAULT_SOUND_VOLUME = 80;

    private GameSettingsManager() { }

    private static SharedPreferences prefs(Context context) {
        return context.getSharedPreferences(PREFS, Context.MODE_PRIVATE);
    }

    public static boolean isMusicEnabled(Context context) { return prefs(context).getBoolean(MUSIC_ENABLED, true); }
    public static int getMusicVolume(Context context) { return prefs(context).getInt(MUSIC_VOLUME, DEFAULT_MUSIC_VOLUME); }
    public static boolean isSoundEnabled(Context context) { return prefs(context).getBoolean(SOUND_ENABLED, true); }
    public static int getSoundVolume(Context context) { return prefs(context).getInt(SOUND_VOLUME, DEFAULT_SOUND_VOLUME); }
    public static boolean isVibrationEnabled(Context context) { return prefs(context).getBoolean(VIBRATION_ENABLED, true); }

    public static void setMusicEnabled(Context context, boolean value) { prefs(context).edit().putBoolean(MUSIC_ENABLED, value).apply(); }
    public static void setMusicVolume(Context context, int value) { prefs(context).edit().putInt(MUSIC_VOLUME, clamp(value)).apply(); }
    public static void setSoundEnabled(Context context, boolean value) { prefs(context).edit().putBoolean(SOUND_ENABLED, value).apply(); }
    public static void setSoundVolume(Context context, int value) { prefs(context).edit().putInt(SOUND_VOLUME, clamp(value)).apply(); }
    public static void setVibrationEnabled(Context context, boolean value) { prefs(context).edit().putBoolean(VIBRATION_ENABLED, value).apply(); }
    public static void reset(Context context) { prefs(context).edit().clear().apply(); }

    private static int clamp(int value) { return Math.max(0, Math.min(100, value)); }
}

```

A.7 com/uccd3223/p1_chai_boon_hong_2206806/HistoryActivity.java

```
package com.uccd3223.p1_chai_boon_hong_2206806;
```

```

import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.google.android.material.button.MaterialButton;

import java.util.ArrayList;
import java.util.List;

```

```

public class HistoryActivity extends AppCompatActivity {
    private static final String STATE_SELECTED_MODE = "history_selected_mode";

    private RecyclerView rvHistory;
    private List<RankedHistoryRecord> historyList;
    private TextView tvEmptyHistory;
    private MaterialButton btnHistoryFun;
    private MaterialButton btnHistoryChallenge;
    private int selectedIndex;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_history);

        findViewById(R.id.btnBack).setOnClickListener(view -> finish());
        rvHistory = findViewById(R.id.rvHistory);
        tvEmptyHistory = findViewById(R.id.tvEmptyHistory);
        rvHistory.setLayoutManager(new LinearLayoutManager(this));
        historyList = HistoryRankingUtil.rankNewestFirst(HistoryManager.getHistory(this));

        btnHistoryFun = findViewById(R.id.btnHistoryFun);
        btnHistoryChallenge = findViewById(R.id.btnHistoryChallenge);
        btnHistoryFun.setOnClickListener(view -> selectMode(0));
        btnHistoryChallenge.setOnClickListener(view -> selectMode(1));
        selectMode(savedInstanceState == null
            ? 0
            : savedInstanceState.getInt(STATE_SELECTED_MODE, 0));
    }

    private void selectMode(int modeIndex) {
        selectedIndex = Math.max(0, Math.min(1, modeIndex));
        btnHistoryFun.setChecked(selectedModeIndex == 0);
        btnHistoryChallenge.setChecked(selectedModeIndex == 1);
        updateList(selectedModeIndex);
    }

    private void updateList(int tabPosition) {
        List<RankedHistoryRecord> filtered = new ArrayList<>();
        for (RankedHistoryRecord ranked : historyList) {
            HistoryRecord record = ranked.getRecord();
            boolean funRecord = "FUN".equals(record.getMode());
            if (tabPosition == 0 && funRecord || tabPosition == 1 && !funRecord) {
                filtered.add(ranked);
            }
        }
        rvHistory.setAdapter(new HistoryAdapter(filtered));
        tvEmptyHistory.setVisibility(filtered.isEmpty() ? View.VISIBLE : View.GONE);
    }

    @Override
    protected void onSaveInstanceState(Bundle outState) {
        outState.putInt(STATE_SELECTED_MODE, selectedIndex);
        super.onSaveInstanceState(outState);
    }
}

```

A.8 com/uccd3223/p1_chai_boon_hong_2206806/HistoryAdapter.java

```

package com.uccd3223.p1_chai_boon_hong_2206806;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;
import android.widget.ImageView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.google.android.material.chip.Chip;

import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.List;
import java.util.Locale;

public class HistoryAdapter extends RecyclerView.Adapter<HistoryAdapter.ViewHolder> {
    private final List<RankedHistoryRecord> records;

    public HistoryAdapter(List<RankedHistoryRecord> records) {
        this.records = records;
    }
}

```

```

@NonNull
@Override
public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
    View view = LayoutInflater.from(parent.getContext()).inflate(
        R.layout.item_history,
        parent,
        false
    );
    return new ViewHolder(view);
}

@Override
public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
    RankedHistoryRecord ranked = records.get(position);
    HistoryRecord record = ranked.getRecord();
    holder.ivHistoryIcon.setImageResource(ranked.isHighest() ? R.drawable.ic_trophy_gold : R.drawable.ic_history_clock);
    holder.ivHistoryIcon.setContentDescription(holder.itemView.getContext().getString(
        ranked.isHighest() ? R.string.highest_score : R.string.previous_result));
    switch (record.getGameName()) {
        case "Association":
            holder.tvGameName.setText(R.string.number_association_title);
            break;
        case "PlaceValue":
            holder.tvGameName.setText(R.string.place_value_title);
            break;
        case "Recognition":
            holder.tvGameName.setText(R.string.number_recognition_title);
            break;
        case "Sequence":
            holder.tvGameName.setText(R.string.number_sequence_title);
            break;
        default:
            holder.tvGameName.setText(record.getGameName());
            break;
    }

    if ("FUN".equals(record.getMode())) {
        holder.chipMode.setVisibility(View.GONE);
    } else {
        holder.chipMode.setVisibility(View.VISIBLE);
        if ("TIME_ATTACK".equals(record.getMode())) {
            if (record.getTimeLimitMs() > 0L) {
                holder.chipMode.setText(holder.itemView.getContext().getString(
                    R.string.mode_time_attack_duration, record.getTimeLimitMs() / 1_000L));
            } else {
                holder.chipMode.setText(R.string.mode_time_attack);
            }
        } else if ("SCORE_ATTACK".equals(record.getMode())) {
            holder.chipMode.setText(R.string.mode_score_attack);
        } else {
            holder.chipMode.setText(record.getMode());
        }
    }

    if ("SCORE_ATTACK".equals(record.getMode())) {
        holder.tvScore.setText(holder.itemView.getContext().getString(
            R.string.round_reached,
            record.getScore()
        ));
    } else {
        holder.tvScore.setText(holder.itemView.getResources().getQuantityString(
            R.plurals.points_earned,
            record.getScore(),
            record.getScore()
        ));
    }

    SimpleDateFormat format = new SimpleDateFormat("MMM dd, yyyy h:mm a", Locale.getDefault());
    holder.tvDate.setText(format.format(new Date(record.getTimestamp())));
}

@Override
public int getItemCount() {
    return records.size();
}

static final class ViewHolder extends RecyclerView.ViewHolder {
    final TextView tvGameName;
    final Chip chipMode;
    final TextView tvScore;
    final TextView tvDate;
    final ImageView ivHistoryIcon;

    ViewHolder(View view) {
        super(view);
        tvGameName = view.findViewById(R.id.tvGameName);
        chipMode = view.findViewById(R.id.chipMode);
        tvScore = view.findViewById(R.id.tvScore);
        tvDate = view.findViewById(R.id.tvDate);
        ivHistoryIcon = view.findViewById(R.id.trophyIcon);
    }
}

```

```
}
```

A.9 com/uccd3223/p1_chai_boon_hong_2206806/HistoryManager.java

```
package com.uccd3223.p1_chai_boon_hong_2206806;

import android.content.Context;
import android.content.SharedPreferences;

import com.google.gson.Gson;
import com.google.gson.reflect.TypeToken;

import java.lang.reflect.Type;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public final class HistoryManager {
    private static final String PREFS_NAME = "GameHistoryPrefs";
    private static final String KEY_HISTORY = "history_list";
    private static final int MAX_HISTORY_RECORDS = 100;

    private HistoryManager() {
    }

    public static void saveRecord(Context context, HistoryRecord record) {
        SharedPreferences prefs = context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE);
        List<HistoryRecord> history = new ArrayList<>(getHistory(context));
        history.add(0, record);
        if (history.size() > MAX_HISTORY_RECORDS) {
            history.subList(MAX_HISTORY_RECORDS, history.size()).clear();
        }
        prefs.edit().putString(KEY_HISTORY, new Gson().toJson(history)).apply();
    }

    public static List<HistoryRecord> getHistory(Context context) {
        SharedPreferences prefs = context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE);
        String json = prefs.getString(KEY_HISTORY, null);
        if (json == null) return Collections.emptyList();

        Type type = new TypeToken<List<HistoryRecord>>().getType();
        try {
            List<HistoryRecord> history = new Gson().fromJson(json, type);
            return history == null ? Collections.emptyList() : history;
        } catch (RuntimeException exception) {
            return Collections.emptyList();
        }
    }

    public static void reset(Context context) {
        context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE).edit().clear().apply();
    }
}
```

A.10 com/uccd3223/p1_chai_boon_hong_2206806/HistoryRankingUtil.java

```
package com.uccd3223.p1_chai_boon_hong_2206806;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

public final class HistoryRankingUtil {
    private HistoryRankingUtil() { }

    public static List<RankedHistoryRecord> rankNewestFirst(List<HistoryRecord> records) {
        Map<String, Integer> maximums = new HashMap<>();
        for (HistoryRecord record : records) {
            String category = eligibleCategory(record);
            if (category != null) maximums.merge(category, record.getScore(), Math::max);
        }
        Set<String> awarded = new HashSet<>();
        List<RankedHistoryRecord> ranked = new ArrayList<>();
        for (HistoryRecord record : records) {
            String category = eligibleCategory(record);
            boolean highest = category != null
                && record.getScore() == maximums.get(category)
                && awarded.add(category);
            ranked.add(new RankedHistoryRecord(record, highest));
        }
        return ranked;
    }
}
```

```

static String eligibleCategory(HistoryRecord record) {
    if ("TIME_ATTACK".equals(record.getMode()) && record.getTimeLimitMs() <= 0L) return null;
    String category = ProgressManager.category(record.getMode(), record.getTimeLimitMs());
    if ("UNKNOWN".equals(category) || "TA_LEGACY".equals(category)) return null;
    return record.getGameName() + "_" + category;
}
}

```

A.11 com/uccd3223/p1_chai_boon_hong_2206806/HistoryRecord.java

```

package com.uccd3223.p1_chai_boon_hong_2206806;

import java.util.Objects;

public final class HistoryRecord {
    private final String gameName;
    private final String mode;
    private final int score;
    private final long timestamp;
    private final long timeLimitMs;

    public HistoryRecord(String gameName, String mode, int score, long timestamp) {
        this(gameName, mode, score, timestamp, 0L);
    }

    public HistoryRecord(String gameName, String mode, int score, long timestamp, long timeLimitMs) {
        this.gameName = Objects.requireNonNull(gameName);
        this.mode = Objects.requireNonNull(mode);
        this.score = score;
        this.timestamp = timestamp;
        this.timeLimitMs = timeLimitMs;
    }

    public String getGameName() {
        return gameName;
    }

    public String getMode() {
        return mode;
    }

    public int getScore() {
        return score;
    }

    public long getTimestamp() {
        return timestamp;
    }

    public long getTimeLimitMs() { return timeLimitMs; }

    @Override
    public boolean equals(Object other) {
        if (this == other) return true;
        if (!(other instanceof HistoryRecord)) return false;
        HistoryRecord record = (HistoryRecord) other;
        return score == record.score
            && timestamp == record.timestamp
            && timeLimitMs == record.timeLimitMs
            && gameName.equals(record.gameName)
            && mode.equals(record.mode);
    }

    @Override
    public int hashCode() {
        return Objects.hash(gameName, mode, score, timestamp, timeLimitMs);
    }
}

```

A.12 com/uccd3223/p1_chai_boon_hong_2206806/MainActivity.java

```

package com.uccd3223.p1_chai_boon_hong_2206806;

import android.app.Dialog;
import android.content.Intent;
import android.os.Bundle;
import android.view.ViewGroup;
import android.view.Window;
import android.widget.Button;
import android.widget.ImageButton;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

import com.google.android.material.button.MaterialButton;

```

```

import com.google.android.material.card.MaterialCardView;

public class MainActivity extends AppCompatActivity {
    private MaterialButton btnFunMode;
    private MaterialButton btnChallengeMode;
    private TextView tvModeHint;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        btnFunMode = findViewById(R.id.btnFunMode);
        btnChallengeMode = findViewById(R.id.btnChallengeMode);
        tvModeHint = findViewById(R.id.tvModeHint);
        findViewById(R.id.btnHistory).setOnClickListener(view ->
            startActivity(new Intent(this, HistoryActivity.class))
        );
        findViewById(R.id.btnSettings).setOnClickListener(view ->
            startActivity(new Intent(this, SettingsActivity.class))
        );

        btnFunMode.setOnClickListener(view -> selectMode(false));
        btnChallengeMode.setOnClickListener(view -> selectMode(true));
        findViewById(R.id.cardSequence).setOnClickListener(view ->
            handleGameSelection(SequenceActivity.class));
        findViewById(R.id.cardAssociation).setOnClickListener(view ->
            handleGameSelection(AssociationActivity.class));
        findViewById(R.id.cardPlaceValue).setOnClickListener(view ->
            handleGameSelection(PlaceValueActivity.class));
        findViewById(R.id.cardRecognition).setOnClickListener(view ->
            handleGameSelection(RecognitionActivity.class));
    }

    private void selectMode(boolean challenge) {
        btnFunMode.setChecked(!challenge);
        btnChallengeMode.setChecked(challenge);
        tvModeHint.setText(challenge ? R.string.challenge_mode_hint : R.string.fun_mode_hint);
    }

    private String getSelectedMode() {
        return btnChallengeMode.isChecked() ? "CHALLENGE" : "FUN";
    }

    private void handleGameSelection(Class<?> activityClass) {
        if ("FUN".equals(getSelectedMode())) {
            Intent intent = new Intent(this, activityClass);
            intent.putExtra("GAME_MODE", "FUN");
            startActivity(intent);
        } else {
            showChallengeSelectionDialog(activityClass);
        }
    }

    private void showChallengeSelectionDialog(Class<?> activityClass) {
        Dialog dialog = new Dialog(this, R.style.FullScreenDialogTheme);
        dialog.setContentView(R.layout.dialog_challenge_selection);
        dialog.<Button>findViewById(R.id.btnTimeAttack).setOnClickListener(view -> {
            dialog.dismiss();
            showTimeSelectionDialog(activityClass);
        });
        dialog.<Button>findViewById(R.id.btnScoreAttack).setOnClickListener(view -> {
            dialog.dismiss();
            Intent intent = new Intent(this, activityClass);
            intent.putExtra("GAME_MODE", "SCORE_ATTACK");
            startActivity(intent);
        });
        dialog.<ImageButton>findViewById(R.id.btnClose).setOnClickListener(view -> dialog.dismiss());
        showFullScreenDialog(dialog);
    }

    private void showTimeSelectionDialog(Class<?> activityClass) {
        Dialog dialog = new Dialog(this, R.style.FullScreenDialogTheme);
        dialog.setContentView(R.layout.dialog_time_selection);
        dialog.<Button>findViewById(R.id.btn60s).setOnClickListener(view ->
            startTimeAttack(dialog, activityClass, 60_000L));
        dialog.<Button>findViewById(R.id.btn90s).setOnClickListener(view ->
            startTimeAttack(dialog, activityClass, 90_000L));
        dialog.<Button>findViewById(R.id.btn120s).setOnClickListener(view ->
            startTimeAttack(dialog, activityClass, 120_000L));
        dialog.<ImageButton>findViewById(R.id.btnClose).setOnClickListener(view -> dialog.dismiss());
        showFullScreenDialog(dialog);
    }

    private void startTimeAttack(Dialog dialog, Class<?> activityClass, long timeLimit) {
        dialog.dismiss();
        Intent intent = new Intent(this, activityClass);
        intent.putExtra("GAME_MODE", "TIME_ATTACK");
        intent.putExtra("TIME_LIMIT", timeLimit);
        startActivity(intent);
    }
}

```

```

    }

    private void showFullScreenDialog(Dialog dialog) {
        dialog.show();
        Window window = dialog.getWindow();
        if (window != null) {
            window.setBackgroundDrawableResource(android.R.color.transparent);
            window.setLayout(ViewGroup.LayoutParams.MATCH_PARENT, ViewGroup.LayoutParams.MATCH_PARENT);
        }
    }
}

```

A.13 com/uccd3223/p1_chai_boon_hong_2206806/NumberQuestApplication.java

```

package com.uccd3223.p1_chai_boon_hong_2206806;

import android.app.Activity;
import android.app.Application;
import android.os.Bundle;

public class NumberQuestApplication extends Application implements Application.ActivityLifecycleCallbacks {
    private int startedActivities;
    private boolean changingConfiguration;

    @Override public void onCreate() {
        super.onCreate();
        registerActivityLifecycleCallbacks(this);
        GameAudioManager.get(this);
    }

    @Override public void onActivityCreated(Activity activity) {
        if (startedActivities++ == 0 && !changingConfiguration) GameAudioManager.get(this).setAppForeground(true);
        changingConfiguration = false;
    }

    @Override public void onActivityStopped(Activity activity) {
        changingConfiguration = activity.isChangingConfigurations();
        if (--startedActivities == 0 && !changingConfiguration) GameAudioManager.get(this).setAppForeground(false);
    }

    @Override public void onActivityCreated(Activity activity, Bundle state) { }
    @Override public void onActivityResumed(Activity activity) { }
    @Override public void onActivityPaused(Activity activity) { }
    @Override public void onActivitySaveInstanceState(Activity activity, Bundle state) { }
    @Override public void onActivityDestroyed(Activity activity) { }
}

```

A.14 com/uccd3223/p1_chai_boon_hong_2206806/PlaceValueActivity.java

```

package com.uccd3223.p1_chai_boon_hong_2206806;

import android.os.Bundle;
import android.view.View;
import android.widget.GridLayout;
import android.widget.ImageView;
import android.widget.TextView;

import com.google.android.flexbox.FlexboxLayout;
import com.google.android.material.card.MaterialCardView;
import com.uccd3223.p1_chai_boon_hong_2206806.util.ExerciseGeneratorUtil;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Locale;

public class PlaceValueActivity extends BaseGameActivity {
    private static final String STATE_TENS = "place_value_tens";
    private static final String STATE_ONES = "place_value_ones";
    private static final String STATE_LAST_TOTAL = "place_value_last_total";
    private static final String STATE_OPTIONS = "place_value_options";
    private static final String STATE_QUESTION_LOCKED = "place_value_question_locked";

    private FlexboxLayout tensContainerLayout;
    private FlexboxLayout onesContainerLayout;
    private GridLayout optionsContainerLayout;
    private TextView tvTensCount;
    private TextView tvOnesCount;
    private TextView tvFeedback;
    private ExerciseGeneratorUtil.PlaceValueData currentData;
    private List<Integer> currentOptions = Collections.emptyList();
    private int lastTotal = -1;
    private boolean questionLocked;

    @Override
    protected void onCreate(Bundle savedInstanceState) {

```

```

        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_place_value);
        setupGameModeUI();
        initializeUI();
        if (savedInstanceState != null && savedInstanceState.containsKey(STATE_TENS)) {
            restoreQuestion(savedInstanceState);
        } else {
            loadNextQuestion();
        }
    }

private void initializeUI() {
    tensContainerLayout = findViewById(R.id.tensContainerLayout);
    onesContainerLayout = findViewById(R.id.onesContainerLayout);
    optionsContainerLayout = findViewById(R.id.optionsContainerLayout);
    tvTensCount = findViewById(R.id.tvTensCount);
    tvOnesCount = findViewById(R.id.tvOnesCount);
    tvFeedback = findViewById(R.id.tvFeedback);
}

private void loadNextQuestion() {
    questionLocked = false;
    resetFeedback(tvFeedback);
    do {
        currentData = ExerciseGeneratorUtil.generatePlaceValue(9);
    } while (currentData.getTotal() == lastTotal);
    lastTotal = currentData.getTotal();
    currentOptions = ExerciseGeneratorUtil
        .generateRecognitionOptions(currentData.getTotal(), 99, 4)
        .getOptions();
    renderQuestion();
}

private void restoreQuestion(Bundle state) {
    currentData = new ExerciseGeneratorUtil.PlaceValueData(
        state.getInt(STATE_TENS),
        state.getInt(STATE_ONES)
    );
    lastTotal = state.getInt(STATE_LAST_TOTAL, currentData.getTotal());
    questionLocked = state.getBoolean(STATE_QUESTION_LOCKED);
    ArrayList<Integer> restoredOptions = state.getIntegerArrayList(STATE_OPTIONS);
    currentOptions = restoredOptions == null ? Collections.emptyList() : restoredOptions;
    if (currentOptions.size() != 4) {
        currentOptions = ExerciseGeneratorUtil
            .generateRecognitionOptions(currentData.getTotal(), 99, 4)
            .getOptions();
    }
    renderQuestion();
    if (questionLocked) {
        restoreCompletedAnswerChoice(
            optionsContainerLayout,
            currentData.getTotal(),
            tvFeedback,
            successMessage(),
            this::loadNextQuestion
        );
    }
}

private void renderQuestion() {
    renderVisualUI();
    renderOptionsUI();
}

private void renderVisualUI() {
    tensContainerLayout.removeAllViews();
    onesContainerLayout.removeAllViews();
    tvTensCount.setText(getResources().getQuantityString(
        R.plurals.tens_count,
        currentData.getTens(),
        currentData.getTens()
    ));
    tvOnesCount.setText(getResources().getQuantityString(
        R.plurals.ones_count,
        currentData.getOnes(),
        currentData.getOnes()
    ));

    float density = getResources().getDisplayMetrics().density;
    int margin = (int) (3 * density);
    int tensWidth = (int) (18 * density);
    int tensHeight = (int) (118 * density);
    int oneSize = (int) (28 * density);

    for (int index = 0; index < currentData.getTens(); index++) {
        ImageView view = new ImageView(this);
        view.setImageResource(R.drawable.ic_tens_block);
        view.setScaleType(ImageView.ScaleType.FIT_XY);
        view.setContentDescription(getString(R.string.ten_block_description));
        view.setImportantForAccessibility(View.IMPORTANT_FOR_ACCESSIBILITY_YES);
        FlexboxLayout.LayoutParams params = new FlexboxLayout.LayoutParams(tensWidth, tensHeight);
        params.setMargins(margin, margin, margin, margin);
    }
}

```

```

        view.setLayoutParams(params);
        tensContainerLayout.addView(view);
    }

    for (int index = 0; index < currentData.getOnes(); index++) {
        ImageView view = new ImageView(this);
        view.setImageResource(R.drawable.ic_ones_block);
        view.setScaleType(ImageView.ScaleType.FIT_XY);
        view.setContentDescription(getString(R.string.one_block_description));
        view.setImportantForAccessibility(View.IMPORTANT_FOR_ACCESSIBILITY_YES);
        FlexboxLayout.LayoutParams params = new FlexboxLayout.LayoutParams(oneSize, oneSize);
        params.setMargins(margin, margin, margin, margin);
        view.setLayoutParams(params);
        onesContainerLayout.addView(view);
    }
}

private void renderOptionsUI() {
    List<AnswerChoice> choices = new ArrayList<>();
    for (int option : currentOptions) {
        choices.add(new AnswerChoice(
            option,
            getString(R.string.number_value, option),
            getString(R.string.answer_number, option)
        ));
    }
    renderAnswerChoices(optionsContainerLayout, choices, (choice, card) ->
        checkAnswer(choice.getValue(), card));
}

private void checkAnswer(int selectedOption, MaterialCardView card) {
    if (questionLocked || !canAcceptInput()) return;
    if (selectedOption == currentData.getTotal()) {
        questionLocked = true;
        completeAnswerChoice(
            optionsContainerLayout,
            card,
            tvFeedback,
            successMessage(),
            this::loadNextQuestion
        );
    } else {
        retryAnswerChoice(card, tvFeedback, getString(R.string.place_value_retry));
    }
}

private String successMessage() {
    String placeParts = getString(
        R.string.number_words_pair,
        getResources().getQuantityString(
            R.plurals.tens_count,
            currentData.getTens(),
            currentData.getTens()
        ),
        getResources().getQuantityString(
            R.plurals.ones_count,
            currentData.getOnes(),
            currentData.getOnes()
        )
    );
    String numberWords = ExerciseGeneratorUtil.numberToWords(currentData.getTotal())
        .toLowerCase(Locale.getDefault());
    return getString(R.string.place_value_success, placeParts, numberWords);
}

@Override
protected void onSaveInstanceState(Bundle outState) {
    if (currentData != null) {
        outState.putInt(STATE_TENS, currentData.getTens());
        outState.putInt(STATE_ONES, currentData.getOnes());
    }
    outState.putInt(STATE_LAST_TOTAL, lastTotal);
    outState.putIntegerArrayList(STATE_OPTIONS, new ArrayList<>(currentOptions));
    outState.putBoolean(STATE_QUESTION_LOCKED, questionLocked);
    super.onSaveInstanceState(outState);
}
}

```

A.15 com/uccd3223/p1_chai_boon_hong_2206806/ProgressManager.java

```
package com.uccd3223.p1_chai_boon_hong_2206806;
```

```
import android.content.Context;
import android.content.SharedPreferences;
```

```
public final class ProgressManager {
    private static final String Prefs = "GameHighScores";
    public static final int NO_SCORE = -1;

    private ProgressManager() { }
}

```

```

public static String category(String mode, long timeLimitMs) {
    if ("FUN".equals(mode)) return "FUN";
    if ("TIME_ATTACK".equals(mode)) return timeLimitMs > 0 ? "TA_" + timeLimitMs : "TA_LEGACY";
    if ("SCORE_ATTACK".equals(mode)) return "ROUND_RUSH";
    return "UNKNOWN";
}

public static String key(String gameName, String mode, long timeLimitMs) {
    return gameName + "_" + category(mode, timeLimitMs);
}

public static int getBest(Context context, String gameName, String mode, long timeLimitMs) {
    SharedPreferences prefs = context.getSharedPreferences(PREFS, Context.MODE_PRIVATE);
    String scoreKey = key(gameName, mode, timeLimitMs);
    if (prefs.contains(scoreKey)) return prefs.getInt(scoreKey, NO_SCORE);
    if ("SCORE_ATTACK".equals(mode)) return prefs.getInt(gameName + "_SA", NO_SCORE);
    return NO_SCORE;
}

public static boolean updateBest(Context context, String gameName, String mode, long timeLimitMs, int score) {
    SharedPreferences prefs = context.getSharedPreferences(PREFS, Context.MODE_PRIVATE);
    String key = key(gameName, mode, timeLimitMs);
    int previous = getBest(context, gameName, mode, timeLimitMs);
    if (score <= previous) return false;
    prefs.edit().putInt(key, score).apply();
    return true;
}

public static void reset(Context context) {
    context.getSharedPreferences(PREFS, Context.MODE_PRIVATE).edit().clear().apply();
}
}

```

A.16 com/uccd3223/p1_chai_boon_hong_2206806/RankedHistoryRecord.java

```

package com.uccd3223.p1_chai_boon_hong_2206806;

public final class RankedHistoryRecord {
    private final HistoryRecord record;
    private final boolean highest;

    public RankedHistoryRecord(HistoryRecord record, boolean highest) {
        this.record = record;
        this.highest = highest;
    }

    public HistoryRecord getRecord() { return record; }
    public boolean isHighest() { return highest; }
}

```

A.17 com/uccd3223/p1_chai_boon_hong_2206806/RecognitionActivity.java

```

package com.uccd3223.p1_chai_boon_hong_2206806;

import android.os.Bundle;
import android.util.TypedValue;
import android.widget.GridLayout;
import android.widget.TextView;

import com.google.android.material.card.MaterialCardView;
import com.uccd3223.p1_chai_boon_hong_2206806.util.ExerciseGeneratorUtil;
import com.uccd3223.p1_chai_boon_hong_2206806.util.ExerciseGeneratorUtil.RecognitionPromptMode;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class RecognitionActivity extends BaseGameActivity {
    private static final String STATE_TARGET = "recognition_target";
    private static final String STATE_LAST_TARGET = "recognition_last_target";
    private static final String STATE_PROMPT_MODE = "recognition_prompt_mode";
    private static final String STATE_LAST_MODE = "recognition_last_mode";
    private static final String STATE_OPTIONS = "recognition_options";
    private static final String STATE_QUESTION_LOCKED = "recognition_question_locked";

    private TextView tvTarget;
    private TextView tvTargetPrompt;
    private GridLayout optionsContainerLayout;
    private TextView tvFeedback;
    private int targetNumber;
    private List<Integer> currentOptions = Collections.emptyList();
    private RecognitionPromptMode promptMode = RecognitionPromptMode.WORD_TO_NUMBER;
    private int lastTarget = -1;
    private int lastModeOrdinal = -1;
    private boolean questionLocked;
}

```

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_recognition);
    setupGameModeUI();
    initializeUI();
    if (savedInstanceState != null && savedInstanceState.containsKey(STATE_TARGET)) {
        restoreQuestion(savedInstanceState);
    } else {
        loadNextQuestion();
    }
}

private void initializeUI() {
    tvTarget = findViewById(R.id.tvTarget);
    tvTargetPrompt = findViewById(R.id.tvTargetPrompt);
    optionsContainerLayout = findViewById(R.id.optionsContainerLayout);
    tvFeedback = findViewById(R.id.tvFeedback);
}

private void loadNextQuestion() {
    questionLocked = false;
    resetFeedback(tvFeedback);
    ExerciseGeneratorUtil.RecognitionExercise exercise;
    do {
        exercise = ExerciseGeneratorUtil.generateRecognitionExercise();
    } while (exercise.getTargetNumber() == lastTarget
        && exercise.getPromptMode().ordinal() == lastModeOrdinal);

    targetNumber = exercise.getTargetNumber();
    promptMode = exercise.getPromptMode();
    lastTarget = targetNumber;
    lastModeOrdinal = promptMode.ordinal();
    currentOptions = exercise.getOptions();
    renderQuestion();
}

private void restoreQuestion(Bundle state) {
    targetNumber = state.getInt(STATE_TARGET);
    lastTarget = state.getInt(STATE_LAST_TARGET, targetNumber);
    int modeIndex = Math.max(
        0,
        Math.min(RecognitionPromptMode.values().length - 1, state.getInt(STATE_PROMPT_MODE))
    );
    promptMode = RecognitionPromptMode.values()[modeIndex];
    lastModeOrdinal = state.getInt(STATE_LAST_MODE, promptMode.ordinal());
    questionLocked = state.getBoolean(STATE_QUESTION_LOCKED);
    ArrayList<Integer> restoredOptions = state.getIntegerArrayList(STATE_OPTIONS);
    currentOptions = restoredOptions == null ? Collections.emptyList() : restoredOptions;
    if (currentOptions.size() != 4) {
        currentOptions = ExerciseGeneratorUtil
            .generateRecognitionOptions(targetNumber, 99, 4)
            .getOptions();
    }
    renderQuestion();
    if (questionLocked) {
        restoreCompletedAnswerChoice(
            optionsContainerLayout,
            targetNumber,
            tvFeedback,
            successMessage(),
            this::loadNextQuestion
        );
    }
}

private void renderQuestion() {
    String numberWords = ExerciseGeneratorUtil.numberToWords(targetNumber);
    List<AnswerChoice> choices = new ArrayList<>();
    if (promptMode == RecognitionPromptMode.WORD_TO_NUMBER) {
        tvTargetPrompt.setText(R.string.find_the_number);
        tvTarget.setText(numberWords);
        tvTarget.setTextSize(TypedValue.COMPLEX_UNIT_SP, 38f);
        for (int option : currentOptions) {
            choices.add(new AnswerChoice(
                option,
                getString(R.string.number_value, option),
                getString(R.string.answer_number, option)
            ));
        }
    } else {
        tvTargetPrompt.setText(R.string.find_the_word);
        tvTarget.setText(getString(R.string.number_value, targetNumber));
        tvTarget.setTextSize(TypedValue.COMPLEX_UNIT_SP, 56f);
        for (int option : currentOptions) {
            String words = ExerciseGeneratorUtil.numberToWords(option);
            choices.add(new AnswerChoice(
                option,
                words,
                getString(R.string.answer_text, words)
            ));
        }
    }
}

```

```

    }
    tvTarget.setContentDescription(tvTarget.getText());
    renderAnswerChoices(optionsContainerLayout, choices, (choice, card) ->
        checkAnswer(choice.getValue(), card));
}

private void checkAnswer(int selectedOption, MaterialCardView card) {
    if (questionLocked || !canAcceptInput()) return;
    if (selectedOption == targetNumber) {
        questionLocked = true;
        completeAnswerChoice(
            optionsContainerLayout,
            card,
            tvFeedback,
            successMessage(),
            this::loadNextQuestion
        );
    } else {
        int retryRes = promptMode == RecognitionPromptMode.WORD_TO_NUMBER
            ? R.string.recognition_retry
            : R.string.recognition_retry_number;
        retryAnswerChoice(card, tvFeedback, getString(retryRes));
    }
}

private String successMessage() {
    return getString(
        R.string.recognition_success,
        ExerciseGeneratorUtil.numberToWords(targetNumber),
        getString(R.string.number_value, targetNumber)
    );
}

@Override
protected void onSaveInstanceState(Bundle outState) {
    outState.putInt(STATE_TARGET, targetNumber);
    outState.putInt(STATE_LAST_TARGET, lastTarget);
    outState.putInt(STATE_PROMPT_MODE, promptMode.ordinal());
    outState.putInt(STATE_LAST_MODE, lastModeOrdinal);
    outState.putIntegerArrayList(STATE_OPTIONS, new ArrayList<>(currentOptions));
    outState.putBoolean(STATE_QUESTION_LOCKED, questionLocked);
    super.onSaveInstanceState(outState);
}
}

```

A.18 com/uccd3223/p1_chai_boon_hong_2206806/SequenceActivity.java

```

package com.uccd3223.p1_chai_boon_hong_2206806;

import android.content.ClipData;
import android.content.ClipDescription;
import android.content.res.ColorStateList;
import android.os.Bundle;
import android.view.DragEvent;
import android.view.Gravity;
import android.view.MotionEvent;
import android.view.View;
import android.view.ViewConfiguration;
import android.widget.FrameLayout;
import android.widget.ImageView;
import android.widget.LinearLayout;
import android.widget.TextView;

import androidx.core.content.ContextCompat;

import com.uccd3223.p1_chai_boon_hong_2206806.util.ExerciseGeneratorUtil;
import com.uccd3223.p1_chai_boon_hong_2206806.util.ExerciseGeneratorUtil.SortDirection;
import com.uccd3223.p1_chai_boon_hong_2206806.util.ExerciseGeneratorUtil.SortExercise;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ThreadLocalRandom;

public class SequenceActivity extends BaseGameActivity {
    private interface BalloonViewFactory {
        FrameLayout create(int index, LinearLayout row);
    }

    private static final int EMPTY_POSITION = -1;
    private static final String STATE_OPTIONS = "sequence_options";
    private static final String STATE_DIRECTION = "sequence_direction";
    private static final String STATE_PLACED = "sequence_placed";
    private static final String STATE_LAST_SIGNATURE = "sequence_last_signature";
    private static final String STATE_QUESTION_LOCKED = "sequence_question_locked";

```

```

private LinearLayout sequenceContainerLayout;
private LinearLayout optionsContainerLayout;
private TextView tvStartDirection;
private TextView tvFeedback;
private SortExercise currentExercise;
private List<Integer> placedNumbers = new ArrayList<>();
private String lastSignature = "";
private boolean questionLocked;
private final List<FrameLayout> targetViews = new ArrayList<>();
private final Map<Integer, FrameLayout> sourceViews = new HashMap<>();

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_sequence);
    setupGameModeUI();
    initializeUI();
    if (savedInstanceState != null && savedInstanceState.containsKey(STATE_OPTIONS)) {
        restoreQuestion(savedInstanceState);
    } else {
        loadNextQuestion();
    }
}

private void initializeUI() {
    sequenceContainerLayout = findViewById(R.id.sequenceContainerLayout);
    optionsContainerLayout = findViewById(R.id.optionsContainerLayout);
    tvStartDirection = findViewById(R.id.tvStartDirection);
    tvFeedback = findViewById(R.id.tvFeedback);
}

private void loadNextQuestion() {
    questionLocked = false;
    resetFeedback(tvFeedback);
    do {
        int length = ThreadLocalRandom.current().nextInt(3, 7);
        currentExercise = ExerciseGeneratorUtil.generateSortExercise(length);
    } while (questionSignature(currentExercise).equals(lastSignature));
    lastSignature = questionSignature(currentExercise);
    placedNumbers = new ArrayList<>(Collections.nCopies(currentExercise.getOptions().size(), null));
    renderQuestion();
}

private void restoreQuestion(Bundle state) {
    int[] optionArray = state.getIntArray(STATE_OPTIONS);
    List<Integer> options = new ArrayList<>();
    if (optionArray != null) {
        for (int option : optionArray) options.add(option);
    }
    if (options.size() < 3 || options.size() > 6
        || new HashSet<>(options).size() != options.size()) {
        loadNextQuestion();
        return;
    }

    int directionIndex = Math.max(
        0,
        Math.min(SortDirection.values().length - 1, state.getInt(STATE_DIRECTION))
    );
    currentExercise = new SortExercise(options, SortDirection.values()[directionIndex]);
    int[] restoredArray = state.getIntArray(STATE_PLACED);
    List<Integer> restored = new ArrayList<>();
    if (restoredArray != null) {
        for (int value : restoredArray) restored.add(value == EMPTY_POSITION ? null : value);
    }
    placedNumbers = restored.size() == options.size()
        ? restored
        : new ArrayList<>(Collections.nCopies(options.size(), null));
    String restoredSignature = state.getString(STATE_LAST_SIGNATURE);
    lastSignature = restoredSignature == null ? "" : restoredSignature;
    questionLocked = state.getBoolean(STATE_QUESTION_LOCKED);
    renderQuestion();
    if (questionLocked) {
        restoreCompletedQuestion(tvFeedback, successMessage(), this::loadNextQuestion);
    }
}

private String questionSignature(SortExercise exercise) {
    List<Integer> sorted = exercise.getOptions();
    Collections.sort(sorted);
    StringBuilder signature = new StringBuilder(exercise.getDirection().name()).append(':');
    for (int index = 0; index < sorted.size(); index++) {
        if (index > 0) signature.append(',');
        signature.append(sorted.get(index));
    }
    return signature.toString();
}

private void renderQuestion() {
    tvStartDirection.setText(currentExercise.getDirection() == SortDirection.ASCENDING
        ? R.string.start_with_smallest
        : R.string.start_with_biggest);
}

```

```

        renderSequenceTargets();
        renderAnswerBalloons();
    }

    private void renderSequenceTargets() {
        targetViews.clear();
        renderBalloonRows(
            sequenceContainerLayout,
            currentExercise.getOptions().size(),
            (index, row) -> {
                BalloonFrameLayout targetView = inflateBalloon(row);
                targetViews.add(targetView);
                Integer placed = placedNumbers.get(index);
                if (placed == null) {
                    targetView.findViewById(R.id.balloonText).setText("");
                    targetView.setContentDescription(getString(
                        R.string.position_empty,
                        index + 1,
                        currentExercise.getOptions().size()
                    ));
                    attachDropTarget(targetView, index);
                } else {
                    fillTargetView(targetView, index, placed);
                }
                return targetView;
            }
        );
    }

    private void renderAnswerBalloons() {
        sourceViews.clear();
        List<Integer> options = currentExercise.getOptions();
        renderBalloonRows(
            optionsContainerLayout,
            options.size(),
            (index, row) -> {
                int number = options.get(index);
                BalloonFrameLayout optionView = inflateBalloon(row);
                sourceViews.put(number, optionView);
                optionView.findViewById(R.id.balloonText).setText(
                    getString(R.string.number_value, number)
                );
                optionView.findViewById(R.id.balloonBg).setImageTintList(
                    ColorStateList.valueOf(colorForNumber(number))
                );
                optionView.setContentDescription(getString(R.string.drag_number, number));
                optionView.setVisibility(placedNumbers.contains(number) ? View.INVISIBLE : View.VISIBLE);
                optionView.setOnClickListener(view -> placeByTap(number, optionView));
                attachImmediateDrag(optionView, number);
                return optionView;
            }
        );
    }

    private void renderBalloonRows(
        LinearLayout parent,
        int count,
        BalloonViewFactory factory
    ) {
        parent.removeAllViews();
        List<Integer> pattern = ExerciseGeneratorUtil.balloonRowPattern(count);
        int itemIndex = 0;
        for (int rowSize : pattern) {
            LinearLayout row = new LinearLayout(this);
            row.setOrientation(LinearLayout.HORIZONTAL);
            row.setGravity(Gravity.CENTER);
            row.setLayoutParams(new LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.MATCH_PARENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
            ));
            for (int item = 0; item < rowSize; item++) {
                row.addView(factory.create(itemIndex++, row));
            }
            parent.addView(row);
        }
    }

    private BalloonFrameLayout inflateBalloon(LinearLayout parent) {
        int size = getResources().getDimensionPixelSize(R.dimen.balloon_size);
        int gap = getResources().getDimensionPixelSize(R.dimen.balloon_gap);
        BalloonFrameLayout balloon = (BalloonFrameLayout) getLayoutInflater().inflate(
            R.layout.item_balloon,
            parent,
            false
        );
        LinearLayout.LayoutParams params = new LinearLayout.LayoutParams(size, size);
        params.setMargins(gap, gap, gap, gap);
        balloon.setLayoutParams(params);
        return balloon;
    }

    private void attachDropTarget(FrameLayout targetView, int targetIndex) {
        targetView.setOnDragListener((view, event) -> {
            View source = event.getLocalState() instanceof View

```

```

        ? (View) event.getLocalState()
        : null;
switch (event.getAction()) {
    case DragEvent.ACTION_DRAG_STARTED:
        return event.getClipDescription() != null
            && event.getClipDescription().hasMimeType(ClipDescription.MIMETYPE_TEXT_PLAIN)
            && source != null;
    case DragEvent.ACTION_DRAG_ENTERED:
        view.setAlpha(0.7f);
        return true;
    case DragEvent.ACTION_DRAG_EXITED:
        view.setAlpha(1f);
        return true;
    case DragEvent.ACTION_DROP:
        view.setAlpha(1f);
        Integer number = parseDraggedNumber(event);
        if (number != null
            && source != null
            && placedNumbers.get(targetIndex) == null
            && number.equals(currentExercise.getOrderedNumbers().get(targetIndex))) {
            placeNumber(targetIndex, number, source);
        } else {
            showSequenceRetry(view);
        }
        return true;
    case DragEvent.ACTION_DRAG_ENDED:
        view.setAlpha(1f);
        if (source != null) source.setAlpha(1f);
        return true;
    default:
        return false;
}
});
}

private Integer parseDraggedNumber(DragEvent event) {
    if (event.getClipData() == null || event.getClipData().getItemCount() == 0) return null;
    try {
        return Integer.valueOf(event.getClipData().getItemAt(0).getText().toString());
    } catch (RuntimeException exception) {
        return null;
    }
}

private void attachImmediateDrag(BalloonFrameLayout optionView, int number) {
    int touchSlop = ViewConfiguration.get(this).getScaledTouchSlop();
    float[] down = new float[2];
    boolean[] dragStarted = new boolean[1];

    optionView.setOnTouchListener((view, event) -> {
        switch (event.getActionMasked()) {
            case MotionEvent.ACTION_DOWN:
                down[0] = event.getX();
                down[1] = event.getY();
                dragStarted[0] = false;
                return true;
            case MotionEvent.ACTION_MOVE:
                if (!dragStarted[0]
                    && (Math.abs(event.getX() - down[0]) > touchSlop
                        || Math.abs(event.getY() - down[1]) > touchSlop)) {
                    ClipData dragData = new ClipData(
                        Integer.toString(number),
                        new String[] { ClipDescription.MIMETYPE_TEXT_PLAIN },
                        new ClipData.Item(Integer.toString(number))
                    );
                    dragStarted[0] = view.startDragAndDrop(
                        dragData,
                        new View.DragShadowBuilder(view),
                        view,
                        0
                    );
                    if (dragStarted[0]) {
                        view.setAlpha(0.35f);
                        view.getParent().requestDisallowInterceptTouchEvent(true);
                    }
                }
                return dragStarted[0];
            case MotionEvent.ACTION_UP:
                view.getParent().requestDisallowInterceptTouchEvent(false);
                if (!dragStarted[0]) view.performClick();
                return true;
            case MotionEvent.ACTION_CANCEL:
                view.getParent().requestDisallowInterceptTouchEvent(false);
                view.setAlpha(1f);
                return true;
            default:
                return dragStarted[0];
        }
    });
}

private void placeByTap(int number, View sourceView) {
    if (questionLocked || !canAcceptInput() || sourceView.getVisibility() != View.VISIBLE) return;
    int nextIndex = -1;
    for (int index = 0; index < placedNumbers.size(); index++) {

```

```

        if (placedNumbers.get(index) == null) {
            nextIndex = index;
            break;
        }
    }
    if (nextIndex >= 0 && number == currentExercise.getOrderedNumbers().get(nextIndex)) {
        placeNumber(nextIndex, number, sourceView);
    } else {
        showSequenceRetry(sourceView);
    }
}

private void placeNumber(int targetIndex, int number, View sourceView) {
    if (questionLocked || !canAcceptInput()
        || placedNumbers.get(targetIndex) != null
        || sourceView.getVisibility() != View.VISIBLE) return;
    placedNumbers.set(targetIndex, number);
    fillTargetView(targetViews.get(targetIndex), targetIndex, number);
    sourceView.setVisibility(View.INVISIBLE);

    boolean complete = true;
    for (Integer placed : placedNumbers) {
        if (placed == null) {
            complete = false;
            break;
        }
    }
    if (complete) {
        questionLocked = true;
        celebrateQuestion(
            targetViews.get(targetIndex),
            tvFeedback,
            successMessage(),
            this::loadNextQuestion
        );
    }
}

private String successMessage() {
    return getString(currentExercise.getDirection() == SortDirection.ASCENDING
        ? R.string.sequence_success_small
        : R.string.sequence_success_big);
}

private void fillTargetView(FrameLayout targetView, int index, int number) {
    targetView.setOnDragListener(null);
    targetView.<TextView>findViewById(R.id.balloonText).setText(
        getString(R.string.number_value, number)
    );
    targetView.<ImageView>findViewById(R.id.balloonBg).setImageTintList(
        ColorStateList.valueOf(colorForNumber(number))
    );
    targetView.setContentDescription(getString(
        R.string.position_filled,
        index + 1,
        currentExercise.getOptions().size(),
        number
    ));
}

private void showSequenceRetry(View anchor) {
    int message = currentExercise.getDirection() == SortDirection.ASCENDING
        ? R.string.sequence_retry_small
        : R.string.sequence_retry_big;
    showRetryFeedback(anchor, tvFeedback, getString(message));
}

private int colorForNumber(int number) {
    int[] colors = {
        R.color.game_blue,
        R.color.game_orange,
        R.color.game_purple,
        R.color.game_green,
        R.color.game_balloon_pink,
        R.color.game_star
    };
    int index = Math.max(0, currentExercise.getOptions().indexOf(number));
    return ContextCompat.getColor(this, colors[index % colors.length]);
}

@Override
protected void onSaveInstanceState(Bundle outState) {
    if (currentExercise != null) {
        List<Integer> options = currentExercise.getOptions();
        int[] optionArray = new int[options.size()];
        int[] placedArray = new int[placedNumbers.size()];
        for (int index = 0; index < options.size(); index++) {
            optionArray[index] = options.get(index);
            Integer placed = placedNumbers.get(index);
            placedArray[index] = placed == null ? EMPTY_POSITION : placed;
        }
        outState.putIntArray(STATE_OPTIONS, optionArray);
    }
}

```

```

        outState.putInt(STATE_DIRECTION, currentExercise.getDirection().ordinal());
        outState.putIntArray(STATE_PLACED, placedArray);
    }
    outState.putString(STATE_LAST_SIGNATURE, lastSignature);
    outState.putBoolean(STATE_QUESTION_LOCKED, questionLocked);
    super.onSaveInstanceState(outState);
}
}

```

A.19 com/uccd3223/p1_chai_boon_hong_2206806/SettingsActivity.java

```

package com.uccd3223.p1_chai_boon_hong_2206806;

import android.os.Bundle;
import android.app.Dialog;
import android.view.ViewGroup;
import android.view.Window;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import com.google.android.material.materialswitch.MaterialSwitch;
import com.google.android.material.slider.Slider;

public class SettingsActivity extends AppCompatActivity {
    private MaterialSwitch musicSwitch;
    private MaterialSwitch soundSwitch;
    private MaterialSwitch vibrationSwitch;
    private Slider musicSlider;
    private Slider soundSlider;
    private boolean binding;

    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_settings);
        findViewById(R.id.btnBack).setOnClickListener(view -> finish());
        musicSwitch = findViewById(R.id.switchMusic);
        soundSwitch = findViewById(R.id.switchSound);
        vibrationSwitch = findViewById(R.id.switchVibration);
        musicSlider = findViewById(R.id.sliderMusic);
        soundSlider = findViewById(R.id.sliderSound);
        bindValues();

        musicSwitch.setOnCheckedChangeListener((button, checked) -> {
            if (binding) return;
            GameSettingsManager.setMusicEnabled(this, checked);
            GameAudioManager.get(this).refreshMusic();
        });
        soundSwitch.setOnCheckedChangeListener((button, checked) -> {
            if (!binding) GameSettingsManager.setSoundEnabled(this, checked);
        });
        vibrationSwitch.setOnCheckedChangeListener((button, checked) -> {
            if (!binding) GameSettingsManager.setVibrationEnabled(this, checked);
        });
        musicSlider.addOnChangeListener((slider, value, fromUser) -> {
            if (fromUser) {
                GameSettingsManager.setMusicVolume(this, Math.round(value));
                GameAudioManager.get(this).refreshMusic();
            }
        });
        soundSlider.addOnChangeListener((slider, value, fromUser) -> {
            if (fromUser) GameSettingsManager.setSoundVolume(this, Math.round(value));
        });
        findViewById(R.id.btnReset).setOnClickListener(view -> showResetConfirmation());
    }

    private void bindValues() {
        binding = true;
        musicSwitch.setChecked(GameSettingsManager.isMusicEnabled(this));
        soundSwitch.setChecked(GameSettingsManager.isSoundEnabled(this));
        vibrationSwitch.setChecked(GameSettingsManager.isVibrationEnabled(this));
        musicSlider.setValue(GameSettingsManager.getMusicVolume(this));
        soundSlider.setValue(GameSettingsManager.getSoundVolume(this));
        binding = false;
    }

    private void showResetConfirmation() {
        Dialog dialog = new Dialog(this, R.style.FullScreenDialogTheme);
        dialog.setContentView(R.layout.dialog_confirmation);
        dialog.<TextView>findViewById(R.id.tvConfirmTitle).setText(R.string.reset_title);
        dialog.<TextView>findViewById(R.id.tvConfirmMessage).setText(R.string.reset_message);
        dialog.<Button>findViewById(R.id.btnConfirmPositive).setText(R.string.reset);
        dialog.<Button>findViewById(R.id.btnConfirmNegative).setText(R.string.cancel);
        dialog.findViewById(R.id.btnConfirmNegative).setOnClickListener(view -> dialog.dismiss());
        dialog.findViewById(R.id.btnConfirmPositive).setOnClickListener(view -> {
            HistoryManager.reset(this);
            ProgressManager.reset(this);
            GameSettingsManager.reset(this);
            bindValues();
        });
    }
}

```

```

        GameAudioManager.get(this).refreshMusic();
        Toast.makeText(this, R.string.reset_complete, Toast.LENGTH_SHORT).show();
        dialog.dismiss();
    });
    dialog.show();
    Window window = dialog.getWindow();
    if (window != null) {
        window.setBackgroundDrawableResource(android.R.color.transparent);
        window.setLayout(ViewGroup.LayoutParams.MATCH_PARENT, ViewGroup.LayoutParams.MATCH_PARENT);
    }
}
}

```

A.20 com/uccd3223/p1_chai_boon_hong_2206806/util/ExerciseGeneratorUtil.java

```

package com.uccd3223.p1_chai_boon_hong_2206806.util;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Objects;
import java.util.Set;
import java.util.concurrent.ThreadLocalRandom;

public final class ExerciseGeneratorUtil {
    public enum SortDirection { ASCENDING, DESCENDING }

    public enum RecognitionPromptMode { WORD_TO_NUMBER, NUMBER_TO_WORD }

    public static final class PlaceValueData {
        private final int tens;
        private final int ones;

        public PlaceValueData(int tens, int ones) {
            this.tens = tens;
            this.ones = ones;
        }

        public int getTens() {
            return tens;
        }

        public int getOnes() {
            return ones;
        }

        public int getTotal() {
            return tens * 10 + ones;
        }

        @Override
        public boolean equals(Object other) {
            if (this == other) return true;
            if (!(other instanceof PlaceValueData)) return false;
            PlaceValueData data = (PlaceValueData) other;
            return tens == data.tens && ones == data.ones;
        }

        @Override
        public int hashCode() {
            return Objects.hash(tens, ones);
        }
    }

    public static final class RecognitionData {
        private final int targetNumber;
        private final List<Integer> options;

        public RecognitionData(int targetNumber, List<Integer> options) {
            this.targetNumber = targetNumber;
            this.options = new ArrayList<>(options);
        }

        public int getTargetNumber() {
            return targetNumber;
        }

        public List<Integer> getOptions() {
            return new ArrayList<>(options);
        }

        @Override
        public boolean equals(Object other) {

```

```

        if (this == other) return true;
        if (!(other instanceof RecognitionData)) return false;
        RecognitionData data = (RecognitionData) other;
        return targetNumber == data.targetNumber && options.equals(data.options);
    }

    @Override
    public int hashCode() {
        return Objects.hash(targetNumber, options);
    }
}

public static final class RecognitionExercise {
    private final int targetNumber;
    private final List<Integer> options;
    private final RecognitionPromptMode promptMode;

    public RecognitionExercise(
        int targetNumber,
        List<Integer> options,
        RecognitionPromptMode promptMode
    ) {
        this.targetNumber = targetNumber;
        this.options = new ArrayList<>(options);
        this.promptMode = Objects.requireNonNull(promptMode);
    }

    public int getTargetNumber() {
        return targetNumber;
    }

    public List<Integer> getOptions() {
        return new ArrayList<>(options);
    }

    public RecognitionPromptMode getPromptMode() {
        return promptMode;
    }

    @Override
    public boolean equals(Object other) {
        if (this == other) return true;
        if (!(other instanceof RecognitionExercise)) return false;
        RecognitionExercise exercise = (RecognitionExercise) other;
        return targetNumber == exercise.targetNumber
            && options.equals(exercise.options)
            && promptMode == exercise.promptMode;
    }

    @Override
    public int hashCode() {
        return Objects.hash(targetNumber, options, promptMode);
    }
}

public static final class SortExercise {
    private final List<Integer> options;
    private final SortDirection direction;

    public SortExercise(List<Integer> options, SortDirection direction) {
        this.options = new ArrayList<>(options);
        this.direction = Objects.requireNonNull(direction);
    }

    public List<Integer> getOptions() {
        return new ArrayList<>(options);
    }

    public SortDirection getDirection() {
        return direction;
    }

    public List<Integer> getOrderedNumbers() {
        List<Integer> ordered = new ArrayList<>(options);
        ordered.sort(direction == SortDirection.ASCENDING
            ? Integer::compareTo
            : Collections.reverseOrder());
        return ordered;
    }

    @Override
    public boolean equals(Object other) {
        if (this == other) return true;
        if (!(other instanceof SortExercise)) return false;
        SortExercise exercise = (SortExercise) other;
        return options.equals(exercise.options) && direction == exercise.direction;
    }
}

```

```

    }

    @Override
    public int hashCode() {
        return Objects.hash(options, direction);
    }
}

public static final class SequenceData {
    private final int[] sequence;
    private final int missingIndex;

    public SequenceData(int[] sequence, int missingIndex) {
        this.sequence = sequence.clone();
        this.missingIndex = missingIndex;
    }

    public int[] getSequence() {
        return sequence.clone();
    }

    public int getMissingIndex() {
        return missingIndex;
    }

    public int getMissingValue() {
        return sequence[missingIndex];
    }

    @Override
    public boolean equals(Object other) {
        if (this == other) return true;
        if (!(other instanceof SequenceData)) return false;
        SequenceData data = (SequenceData) other;
        return missingIndex == data.missingIndex && Arrays.equals(sequence, data.sequence);
    }

    @Override
    public int hashCode() {
        return 31 * Arrays.hashCode(sequence) + missingIndex;
    }
}

private ExerciseGeneratorUtil() {
}

public static int generateObjectCount(int maxObjects) {
    if (maxObjects < 1) throw new IllegalArgumentException("maxObjects must be at least 1");
    return ThreadLocalRandom.current().nextInt(1, maxObjects + 1);
}

public static PlaceValueData generatePlaceValue(int maxTens) {
    if (maxTens < 1) throw new IllegalArgumentException("maxTens must be at least 1");
    return new PlaceValueData(
        ThreadLocalRandom.current().nextInt(1, maxTens + 1),
        ThreadLocalRandom.current().nextInt(0, 10)
    );
}

public static RecognitionData generateRecognitionOptions(
    int targetNumber,
    int maxRange,
    int numOptions
) {
    if (maxRange < 0) throw new IllegalArgumentException("maxRange must not be negative");
    if (targetNumber < 0 || targetNumber > maxRange) {
        throw new IllegalArgumentException("targetNumber must be inside maxRange");
    }
    if (numOptions < 1 || numOptions > maxRange + 1) {
        throw new IllegalArgumentException("numOptions must fit inside the learning range");
    }

    Set<Integer> preferred = new LinkedHashSet<>();
    if (targetNumber >= 10) preferred.add((targetNumber % 10) * 10 + targetNumber / 10);
    preferred.add(targetNumber - 10);
    preferred.add(targetNumber + 10);
    preferred.add(targetNumber - 2);
    preferred.add(targetNumber + 2);
    preferred.add(targetNumber - 1);
    preferred.add(targetNumber + 1);

    Set<Integer> options = new LinkedHashSet<>();
    options.add(targetNumber);
    List<Integer> preferredList = new ArrayList<>(preferred);
    Collections.shuffle(preferredList);
    for (int candidate : preferredList) {

```

```

        if (candidate != targetNumber && candidate >= 0 && candidate <= maxRange) {
            if (options.size() < numOptions) options.add(candidate);
        }
    }

    List<Integer> remaining = new ArrayList<>();
    for (int value = 0; value <= maxRange; value++) {
        if (!options.contains(value)) remaining.add(value);
    }
    Collections.shuffle(remaining);
    for (int candidate : remaining) {
        if (options.size() < numOptions) options.add(candidate);
    }

    List<Integer> shuffled = new ArrayList<>(options);
    Collections.shuffle(shuffled);
    return new RecognitionData(targetNumber, shuffled);
}

public static RecognitionExercise generateRecognitionExercise() {
    int target = ThreadLocalRandom.current().nextInt(0, 100);
    RecognitionPromptMode mode = ThreadLocalRandom.current().nextBoolean()
        ? RecognitionPromptMode.WORD_TO_NUMBER
        : RecognitionPromptMode.NUMBER_TO_WORD;
    return generateRecognitionExercise(99, 4, target, mode);
}

public static RecognitionExercise generateRecognitionExercise(
    int maxRange,
    int numOptions,
    int targetNumber,
    RecognitionPromptMode promptMode
) {
    RecognitionData data = generateRecognitionOptions(targetNumber, maxRange, numOptions);
    return new RecognitionExercise(data.getTargetNumber(), data.getOptions(), promptMode);
}

public static SequenceData generateSequence(int start, int step, int length) {
    if (length < 1) throw new IllegalArgumentException("length must be at least 1");
    int[] sequence = new int[length];
    for (int index = 0; index < length; index++) sequence[index] = start + index * step;
    return new SequenceData(sequence, ThreadLocalRandom.current().nextInt(length));
}

public static String numberToWords(int number) {
    if (number == 0) return "Zero";
    if (number < 0 || number > 99) return Integer.toString(number);
    String[] ones = {
        "", "One", "Two", "Three", "Four", "Five", "Six", "Seven", "Eight", "Nine",
        "Ten", "Eleven", "Twelve", "Thirteen", "Fourteen", "Fifteen", "Sixteen",
        "Seventeen", "Eighteen", "Nineteen"
    };
    String[] tens = {
        "", "", "Twenty", "Thirty", "Forty", "Fifty", "Sixty", "Seventy", "Eighty", "Ninety"
    };
    if (number < 20) return ones[number];
    String tensWord = tens[number / 10];
    String onesWord = ones[number % 10];
    return onesWord.isEmpty() ? tensWord : tensWord + "-" + onesWord;
}

public static List<Integer> generateSortSequence(int length) {
    return generateSortSequence(length, 99);
}

public static List<Integer> generateSortSequence(int length, int maxNumber) {
    if (maxNumber < 1) throw new IllegalArgumentException("maxNumber must be at least 1");
    if (length < 1 || length > maxNumber) {
        throw new IllegalArgumentException("length must be between 1 and maxNumber");
    }
    Set<Integer> numbers = new LinkedHashSet<>();
    while (numbers.size() < length) {
        numbers.add(ThreadLocalRandom.current().nextInt(1, maxNumber + 1));
    }
    return new ArrayList<>(numbers);
}

public static SortExercise generateSortExercise(int length) {
    SortDirection direction = ThreadLocalRandom.current().nextBoolean()
        ? SortDirection.ASCENDING
        : SortDirection.DESENDING;
    return generateSortExercise(length, 0, 99, direction);
}

public static SortExercise generateSortExercise(
    int length,
    int minNumber,
    int maxNumber,
    SortDirection direction
) {

```

```

        if (minNumber < 0) throw new IllegalArgumentException("minNumber must not be negative");
        if (maxNumber > 99) throw new IllegalArgumentException("maxNumber must be no more than 99");
        if (minNumber > maxNumber) {
            throw new IllegalArgumentException("minNumber must not exceed maxNumber");
        }
        if (length < 1 || length > maxNumber - minNumber + 1) {
            throw new IllegalArgumentException("length must fit inside the learning range");
        }

        Set<Integer> numbers = new LinkedHashSet<>();
        while (numbers.size() < length) {
            numbers.add(ThreadLocalRandom.current().nextInt(minNumber, maxNumber + 1));
        }
        List<Integer> options = new ArrayList<>(numbers);
        Collections.shuffle(options);
        return new SortExercise(options, direction);
    }

    public static List<Integer> balloonRowPattern(int count) {
        switch (count) {
            case 3:
                return Collections.singletonList(3);
            case 4:
                return Arrays.asList(2, 2);
            case 5:
                return Arrays.asList(3, 2);
            case 6:
                return Arrays.asList(3, 3);
            default:
                throw new IllegalArgumentException("Balloon count must be between 3 and 6");
        }
    }
}

```